# UNIVERSITÀ DEGLI STUDI DI MESSINA

Department of Mathematical, Computer, Physical and Earth Sciences

Bachelor's Degree in Data Analysis - 4th Year

Academic year 2025 - 2026


Subject: Software Engineering

Zemen: Dual-Calendar Collaborative Web Application

Software Engineering Final Project Report


Students:

Aklesia Berihu Mekonnen - 545854

Meron Zemichael Kahsay - 551795

Supervisor:

Prof. Salvatore Distefano

# Table of Contents

# 1. Introduction

Zemen meaning "era" or "time" in Amharic, is a full-stack collaborative web application that bridges two calendar systems: the Ethiopian (Ge'ez) calendar and the widely used Gregorian calendar. The name reflects the core idea of the project: bringing two distinct conceptions of time into a single, unified, and intuitive interface.

The Ethiopian calendar operates on a fundamentally different structure from the Gregorian one. It has 13 months: twelve of 30 days each and a thirteenth month (Pagume) of 5 or 6 days depending on the Ethiopian leap year cycle. Its New Year falls on September 11 or 12 in the Gregorian calendar, and the Ethiopian year runs approximately 7–8 years behind the Gregorian count. Managing dates, events, and collaborative scheduling across these two systems introduces genuine algorithmic complexity.

Zemen was proposed as a dual-calendar event management platform, but through the development process, and driven by the professor's challenge to go beyond basic CRUD functionality, it evolved into something significantly richer:

- A concurrent collaborative editing model using optimistic locking and version conflict detection, ensuring safe simultaneous edits by multiple users.
- A multi-stage smart scheduling engine based on interval merging, gap finding, candidate slot generation, and constraint-based ranking.
- A full field-level change-tracking system with immutable event snapshots, structured plain-text change summaries, and a visual diff viewer.
- Google OAuth 2.0 integration for authentication and Google Calendar export.
- Email reminder delivery via a background daemon thread with duplicate-prevention logging.

The system is deployed using Docker Compose, allowing the entire three-tier stack, React frontend, FastAPI backend, and PostgreSQL database, to launch with a single command. It is comprehensively tested with 116 Pytest unit tests (all passing) and 93 Postman integration tests (all passing).

This report documents the full software engineering journey of Zemen: the methodology we followed, every sprint in detail, the architecture and design decisions we made, the algorithms we implemented, the testing strategy we applied, and the lessons we learned.

# 2. General Existing Software Inspection and Motivation

Before writing a single line of code, we examined the landscape of existing calendar and scheduling tools to understand what they do well and where they fall short for our target users, Ethiopian students studying abroad, international teams collaborating with Ethiopian partners, and anyone who needs to plan around the Ethiopian calendar.

## 2.1 Problems in Existing Solutions

#### No Dual-Calendar Support

Every major calendar application, like Google Calendar, Apple Calendar, Outlook assumes the Gregorian calendar as the sole reference. There is no built-in support for the Ethiopian calendar. Users who need to track Ethiopian holidays, schedule around Timkat or Enkutatash, or communicate dates with Ethiopian partners must perform manual conversion every time.

#### No Integrated Conversion Tool

Standalone Ethiopian-Gregorian converters exist as simple web pages, but they are entirely disconnected from any scheduling or event management functionality. A user must convert a date in one tab, then manually enter it into a calendar in another, a fragmented workflow with high error risk.

#### No Collaborative Conflict Detection

Existing shared calendar tools do not expose version conflict information to users. When two people edit the same event simultaneously, one person's changes are silently overwritten. There is no version history, no field-level diff, and no notification to the losing editor.

#### No Constraint-Based Smart Scheduling Across Calendar Systems

Scheduling tools like Calendly or Doodle find available slots, but they operate entirely in the Gregorian system, have no awareness of Ethiopian public holidays, and provide no ranked output that explains why a slot is better or worse than another.

## 2.2 How Zemen Addresses These Gaps

| Problem Identified | Zemen's Solution |
| --- | --- |

| | |
|---|---|
| No dual-calendar support | Native Ethiopian and Gregorian display; events stored in UTC, shown in either system on demand |
| No integrated conversion tool | Dedicated /convert endpoints and a Convert page in the UI |
| Silent overwrites in collaboration | Optimistic locking with version fields; 409 Conflict responses; field-level diff viewer |
| No smart scheduling across systems | Multi-stage scheduling pipeline with interval merging, gap finding, and constraint-based slot ranking |
| No Ethiopian holiday awareness | holidays table seeded with Ethiopian and Gregorian public holidays; treated as all-day busy blocks in scheduling |

# 3. Agile Methodology

Zemen was developed using the Agile methodology. This section explains what Agile is, its core values and principles, why we chose it over alternative approaches, and the different Agile methods that exist.

## 3.1 Agile Manifesto – 4 Core Values

The Agile Manifesto, published in 2001, defines four core values that guide all Agile methodologies:

| Value | We Value More | Over |
|---|---|---|
| 1 | Individuals and interactions | Processes and tools |
| 2 | Working software | Comprehensive documentation |
| 3 | Customer collaboration | Contract negotiation |
| 4 | Responding to change | Following a plan |

In the context of Zemen, these values shaped our working style directly. We prioritised a working, deployable system at the end of each sprint over writing documentation first. When the professor asked us to extend the system with algorithmically richer features after Sprint 1, we responded to that change rather than strictly following the original plan.

## 3.2 Agile Principles

The Agile Manifesto defines 12 principles. The most relevant to our project were:

1. Early and continuous delivery of working software: every sprint produced a runnable increment.
2. Welcome changing requirements, even late in development: we added optimistic locking and the scheduling engine after Sprint 1 based on feedback.
3. Working software is the primary measure of progress: we tracked progress by passing tests and deployed features, not by document completion.

4. Simplicity, the art of maximising the amount of work not done is essential: we designed the schema in Sprint 1, and it held for the entire project without structural changes, a direct result of keeping it simple but correct.
5. At regular intervals, the team reflects on how to become more effective: our sprint retrospectives identified the FastAPI header issue and the Postman version-staleness problem early.

## 3.3 Why Agile Over a Plan-Driven Approach

A plan-driven (Waterfall) approach would have required us to fully specify all requirements before writing code. For Zemen, this was not suitable for several reasons:

| Dimension | Plan-Driven (Waterfall) | Agile (chosen) |
|---|---|---|
| Requirements | Fixed before development begins | Evolved sprint-by-sprint based on feedback |
| Change handling | Expensive; requires re-planning | Built-in: responding to change is a core value |
| Deliverables | Single large release at the end | Working increments every sprint |
| Risk | High - problems discovered late | Low - issues found and fixed within each sprint |
| Feedback loop | Long -professor sees output at the end | Short - professor could see running system each sprint |
| Team size fit | Better for large, distributed teams | Ideal for a 2-person student team |

The professor's mid-project request to add collaborative features and a scheduling engine is precisely the kind of change that Agile accommodates naturally. Under Waterfall, this would have required restarting the design phase.

## 3.4 Different Types of Agile Methods

Agile is a family of methodologies. The main ones considered were:

| Method | Description | Suitable For |
|---|---|---|
| Scrum | Time-boxed sprints, defined roles (PO, SM, Dev), fixed ceremonies | Projects with evolving requirements; team projects |
| Kanban | Continuous flow, work-in-progress limits, no fixed iteration length | Maintenance or operations work; single-person teams |
| XP (Extreme Programming) | Pair programming, TDD, continuous integration, frequent releases | High-quality code focus; pairs and small teams |
| SAFe | Scaled Agile for large enterprises with multiple teams | Large organizations: not suitable for student projects |
| Lean | Eliminate waste, deliver fast, decide as late as possible | Manufacturing-origin; generic process improvement |

We chose Scrum, as detailed in the next section.

# 4. Scrum Overview

Scrum is an Agile framework for managing and completing complex projects. It organizes work into short, fixed-length iterations called sprints, typically 1–4 weeks long, with defined roles, ceremonies, and artifacts.

## 4.1 Scrum Roles

| Role | Responsibility | In Zemen |
|---|---|---|
| Product Owner (PO) | Defines and prioritises the product backlog; represents stakeholder needs | Shared by both team members |
| Scrum Master (SM) | Facilitates the Scrum process; removes impediments; coaches the team | Rotated between Aklesia and Meron each sprint |
| Development Team | Self-organising team that delivers the sprint increment | Aklesia Berihu Mekonnen & Meron Zemichael Kahsay |

## 4.2 Scrum Artifacts

| Artifact | Description | Zemen Implementation |
|---|---|---|
| Product Backlog | Ordered list of all desired features and requirements | Maintained as a prioritised table covering all 16 deliverables |
| Sprint Backlog | Subset of the Product Backlog selected for the current sprint | Defined at the start of each sprint (see Section 7) |
| Increment | Working software delivered at the end of each sprint | Deployed via Docker Compose; tested via Pytest and Postman |
| Definition of Done (DoD) | Agreement on what 'complete' means for any item | Code written, tests passing, endpoint documented in Postman |

## 4.3 Why Scrum Over Other Agile Frameworks

| Criterion | Scrum | Kanban | XP |
|---|---|---|---|
| Iteration structure | Fixed sprints (ideal for project deadlines) | No fixed iterations | Fixed iterations |
| Planning ceremonies | Sprint planning, review, retrospective | Minimal | Extensive (pair programming, TDD) |
| Progress visibility | Burndown/burnup charts per sprint | Flow metrics | Velocity charts |
| Team size fit | 2–9 people (perfect for us) | Any size | Small teams |
| Learning curve | Moderate - clear structure | Low | High |

Scrum's fixed sprint cadence was particularly valuable because it gave us a clear deadline and review point every week, preventing scope creep and keeping both team members synchronized.

## 4.4 Product Backlog

The full product backlog for Zemen, ordered by priority:

| ID | User Story | Priority | Story Points | Sprint |
|---|---|---|---|---|
| PB-01 | As a user, I can register and log in securely | Must Have | 5 | 1 |
| PB-02 | As a user, I can log in with my Google account | Must Have | 8 | 1 |

| | | | | |
|---|---|---|---|---|
| PB-03 | As a user, I can view and edit my profile and preferences | Must Have | 3 | 1 |
| PB-04 | As a user, I can convert any date between Gregorian and Ethiopian | Must Have | 8 | 2 |
| PB-05 | As a user, I can create, read, update, and delete calendar events | Must Have | 8 | 2 |
| PB-06 | As a user, I can enter event dates in either calendar system | Must Have | 5 | 2 |
| PB-07 | As a user, I can view Ethiopian and Gregorian public holidays | Should Have | 3 | 2 |
| PB-08 | As a user, I can share events with others and assign roles | Must Have | 8 | 3 |
| PB-09 | As an editor, my simultaneous changes are detected with version conflicts | Must Have | 13 | 3 |
| PB-10 | As a user, I can see a field-level diff between two event versions | Should Have | 8 | 3 |
| PB-11 | As an organiser, I can find the best meeting time for all participants | Must Have | 13 | 3 |
| PB-12 | As a user, I can receive ranked time slot suggestions with scores | Should Have | 8 | 3 |
| PB-13 | As a user, I receive email reminders before events | Should Have | 5 | 3 |
| PB-14 | As a user, I can export an event to Google Calendar | Could Have | 5 | 4 |
| PB-15 | All features are covered by Pytest unit tests (target: 100+) | Must Have | 8 | 4 |
| PB-16 | All API endpoints are covered by Postman integration tests (target: 90+) | Must Have | 8 | 4 |

## 4.5 Product Refinement

**Definition of Ready (DoR) for a Story**

- The user story is written in standard format: As a [user], I can [action] so that [value].
- Acceptance criteria are defined and agreed upon by both team members.
- Dependencies on other stories or infrastructure are identified.
- The story has been estimated in story points using planning poker.

**Definition of Done (DoD)**

- Code is written and committed to the main branch.
- At least one Pytest test covers the feature's core logic.
- The relevant API endpoint is documented and passing in the Postman collection.
- The feature is manually verified, running in Docker Compose.

**Cadence and Participants**

Refinement sessions were held at the start of each sprint (Monday) and mid-sprint (Wednesday). Both team members participated. Each session lasted approximately 45–60 minutes.

## 4.6 How We Adhered to Scrum

| Scrum Ceremony | Our Practice |
|---|---|
| Sprint Planning | Held at the start of each sprint; selected backlog items; estimated in story points; defined sprint goal |
| Daily Standup | Informal daily check-in between Aklesia and Meron: what did I do, what will I do? any blockers |
| Sprint Review | End-of-sprint demo of the running system; verified all DoD criteria; updated completion table |
| Sprint Retrospective | Identified what went well and what to improve; documented in each sprint section below |
| Backlog Refinement | Mid-sprint grooming to prepare upcoming stories; adjusted estimates based on lessons learned |

# 5. Software Requirements Analysis

## 5.1 Functional Requirements

The following functional requirements were identified during Sprint 1 planning and refined throughout the project.

**FR-1: User Registration and Authentication**

| Field | Detail |
|---|---|
| ID | FR-1 |
| Description | Users must be able to register with an email and password and log in to receive a JWT access token valid for 60 minutes. |
| Acceptance Criteria | Registration stores bcrypt-hashed passwords. Login returns access_token. Unauthenticated requests receive 401. |
| Priority | Must Have |
| Implemented | Yes - /auth/register, /auth/login |

**FR-2: Google OAuth 2.0 Login**

| Field | Detail |
|---|---|
| ID | FR-2 |
| Description | Users may authenticate via Google OAuth 2.0 as an alternative to email/password registration. |
| Acceptance Criteria | GET /auth/google/login-url redirects to Google. Callback at /auth/google/callback exchanges code for a token and creates or retrieves a user account. |
| Priority | Must Have |
| Implemented | Yes - /auth/google/* router |

**FR-3: User Profile Management**

| Field | Detail |
|---|---|
| ID | FR-3 |
| Description | Users can view and update their profile, including full name, preferred calendar (Ethiopian or Gregorian), time zone, and language. |
| Acceptance Criteria | GET /profile returns current data. PUT /profile updates and returns the updated profile. |
| Priority | Must Have |
| Implemented | Yes - /profile router |

**FR-4: Ethiopian–Gregorian Date Conversion**

| Field | Detail |
|---|---|
| ID | FR-4 |
| Description | The system must convert dates in both directions between the Ethiopian and Gregorian calendar systems, correctly handling the 13th month (Pagume), leap years, and New Year boundary dates. |
| Acceptance Criteria | GET /convert/g2e and /convert/e2g return correct converted dates. Pagume 6 is only valid in Ethiopian leap years. The New Year boundary (Sep 11 vs Sep 12) is correctly computed. |
| Priority | Must Have |
| Implemented | Yes - DataConversionService + /convert router |

**FR-5: Event CRUD**

| Field | Detail |
|---|---|
| ID | FR-5 |

| | |
|---|---|
| Description | Authenticated users can create, list, retrieve, update, and delete calendar events. Events support simple recurrence (daily/weekly), full-text search, and date-range filtering. |
| Acceptance Criteria | POST /events creates. GET /events lists with optional filters. PUT /events/{id} updates with version. DELETE /events/{id} removes. All operations require authentication. |
| Priority | Must Have |
| Implemented | Yes - /events router |

**FR-6: Holiday Data Management**

| Field | Detail |
|---|---|
| ID | FR-6 |
| Description | Public holidays for both calendar systems are stored and displayed. Users can create and delete custom holidays. |
| Acceptance Criteria | Default holidays are seeded at startup. GET /holidays lists all. POST /holidays creates a custom holiday. DELETE /holidays/{id} removes it. |
| Priority | Should Have |
| Implemented | Yes - /holidays router |

**FR-7: Shared Events with Role-Based Access**

| Field | Detail |
|---|---|
| ID | FR-7 |
| Description | Event owners can share events with other users, assigning roles: owner, editor, or viewer. Role permissions are enforced at every relevant endpoint. |
| Acceptance Criteria | POST /events/{id}/share assigns participant and role. Editors can modify. Viewers receive 403 on modification attempts. |
| Priority | Must Have |

| | |
|---|---|
| Implemented | Yes - event_participants table + share endpoint |

**FR-8: Optimistic Locking and Version Conflict Detection**

| Field | Detail |
|---|---|
| ID | FR-8 |
| Description | Each event carries a version integer. Update requests must include the current version. If another user has saved a newer version, the system returns a 409 Conflict response. |
| Acceptance Criteria | Submitting a stale version returns 409 with current_version and code: VERSION_CONFLICT. The correct version proceeds; version field increments; a snapshot is written. |
| Priority | Must Have |
| Implemented | Yes - version field in events table + conflict check in PUT /events/{id} |

**FR-9: Field-Level Diff and Version History**

| Field | Detail |
|---|---|
| ID | FR-9 |
| Description | The system maintains an immutable snapshot of each event version. Users can request a structured field-level diff between any two versions. |
| Acceptance Criteria | GET /events/{id}/diff?from_version=X&to_version=Y returns changed fields with before/after values and a plain-text digest listing the changed field names. |
| Priority | Should Have |
| Implemented | Yes - event_snapshots table + DiffService |

**FR-10: Smart Scheduling — Interval Merging and Slot Ranking**

| Field | Detail |
|---|---|

| | |
|---|---|
| ID | FR-10 |
| Description | Given a search window and duration, the system merges all participant busy intervals, finds free gaps, generates candidate slots, and ranks them by a penalty-based scoring function. |
| Acceptance Criteria | GET /events/{id}/rank returns ranked_slots sorted by score ascending. Required participant conflicts exclude the slot entirely. Optional participant conflicts add +100 penalty. Work-hours violations add +30 penalty. |
| Priority | Must Have |
| Implemented | Yes - IntervalService + RankingService |

**FR-11: Email Reminders**

| Field | Detail |
|---|---|
| ID | FR-11 |
| Description | A background thread polls for events whose reminder window is approaching and sends an email via SMTP. Each reminder is logged to prevent duplicates. |
| Acceptance Criteria | Email is sent once per event reminder. Duplicate send is prevented by reminder_logs table. Thread runs as daemon and does not block application startup. |
| Priority | Should Have |
| Implemented | Yes - ReminderService (daemon thread) |

**FR-12: Google Calendar Export**

| Field | Detail |
|---|---|
| ID | FR-12 |
| Description | Users with a connected Google account can push a Zemen event directly to their Google Calendar. |

| | |
|---|---|
| Acceptance Criteria | POST /google/export-event/{id} creates the event in the connected Google Calendar. Returns the Google Calendar event URL. |
| Priority | Could Have |
| Implemented | Yes - /google router |

## 5.2 Non-Functional Requirements

| ID | Category | Requirement | Implementation |
|---|---|---|---|
| NFR-1 | Security | Passwords must be hashed and never stored in plaintext | bcrypt hashing via passlib |
| NFR-2 | Security | JWTs must expire after a configurable period (default 60 min) | JWT expiry set in SecurityService |
| NFR-3 | Security | SQL injection must be prevented | All queries use SQLAlchemy ORM; no raw SQL strings |
| NFR-4 | Security | Role-based access must be enforced at every endpoint | Dependency injection checks role before each operation |
| NFR-5 | Reliability | The system must degrade gracefully if SMTP or Google OAuth are not configured | try/except guards around all external service calls |
| NFR-6 | Portability | The system must run on any machine with Docker installed | Full Docker Compose configuration with health checks |
| NFR-7 | Data Integrity | All event times must be stored in UTC | start_time_utc column; timezone applied only on output |
| NFR-8 | Testability | All core algorithms must have unit test coverage | 116 Pytest tests across conversion and scheduling |
| NFR-9 | Maintainability | Business logic must be separated from API routing | 6 service classes; routers stay thin |

| | | | |
|---|---|---|---|
| NFR-10 | Performance | Scheduling engine must complete within 2 seconds for typical inputs | Verified in Postman; average response 106ms |

## 5.3 Conceptual Architecture

The conceptual architecture of Zemen is a three-tier client-server model, chosen for its clear separation of concerns, independent deployability of each tier, and proven suitability for web applications with multi-user access patterns.

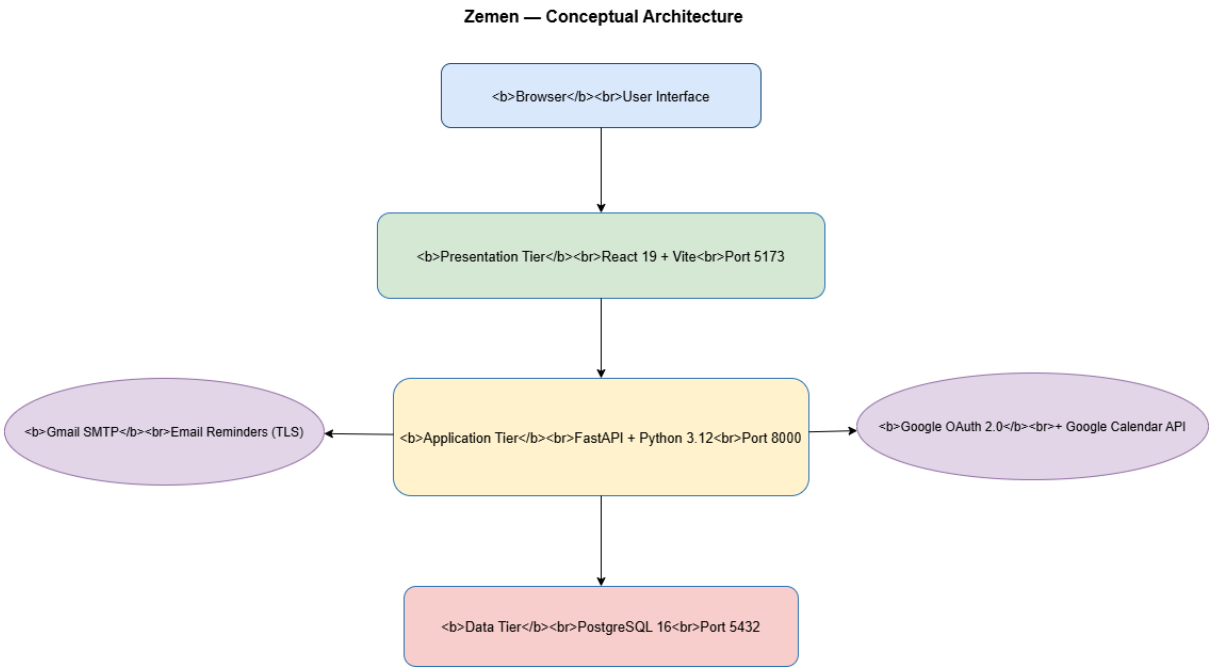


*Figure 1: Zemen Conceptual Architecture - Three-tier client-server model*

The three tiers interact as follows: the browser-based React client communicates exclusively with the FastAPI backend via HTTP/JSON. The backend contains all business logic and is the sole client of the PostgreSQL database. External services (Google OAuth 2.0 and SMTP) are called only by the backend, never directly by the frontend.

# 6. Tools and Technologies

## 6.1 Frontend Technologies

| Technology | Version | Role | Why Chosen |
|---|---|---|---|
| React | 19 | UI component framework | Component model enables reusable calendar, form, and diff-viewer widgets; large ecosystem; team proficiency |
| Vite | 7 | Build tool and dev server | Significantly faster hot-reload than Create React App; native ES module support |
| React Router v7 | Latest | Client-side routing | Declarative routing with nested layouts; RequireAuth wrapper for protected routes |
| Fetch API (native) | Native | HTTP client | Used via custom api.js module; centralized token injection via authHeaders() |

## 6.2 Backend Technologies

| Technology | Version | Role | Why Chosen |
|---|---|---|---|
| FastAPI | Latest | Web framework | Python-native; automatic OpenAPI docs; async-ready; type-safe with Pydantic; excellent for API development |
| Python | 3.12 | Backend language | Strong ecosystem for date arithmetic and data manipulation; well-suited for algorithmic work (scheduling engine) |
| SQLAlchemy | 2.x | ORM | Prevents SQL injection; database-agnostic; clean model definitions; built-in relationship handling |

| | | | |
|---|---|---|---|
| Pydantic | v2 | Data validation | Request/response schema validation integrated with FastAPI; automatic 422 errors for malformed inputs |
| Passlib / bcrypt | Latest | Password hashing | Industry-standard; bcrypt's adaptive cost factor resists brute-force attacks |
| python-jose | Latest | JWT management | Simple, well-maintained; supports HS256 signing with configurable expiry |
| ethiopian_date | Latest | Calendar conversion | Core library for Ethiopian calendar arithmetic; extended with our own wrapper for edge cases |
| smtplib | stdlib | Email delivery | Standard library; no dependency overhead; TLS support built in |

### 6.3 Database

| Technology | Version | Role | Why Chosen |
|---|---|---|---|
| PostgreSQL | 16 | Primary database | ACID compliance essential for optimistic locking; rich data types; reliable at scale; strong SQLAlchemy support |

### 6.4 DevOps and Infrastructure

| Technology | Role | Why Chosen |
|---|---|---|
| Docker | Container runtime for all three tiers | Ensures identical environment on any machine; eliminates 'works on my machine' problems |
| Docker Compose | Multi-container orchestration | Single-command startup (docker compose up --build); service dependency management via health checks |

| | | |
|---|---|---|
| Git | Version control | Full commit history; enables branch-per-sprint workflow |

## 6.5 Testing Tools

| Technology | Role | Coverage |
|---|---|---|
| Pytest | Unit and integration test framework | 116 tests: date conversion edge cases + full scheduling pipeline |
| Postman | API integration testing | 93 tests across 11 folders; fully rerunnable collection |

## 6.6 External Services

| Service | Role | Integration Point |
|---|---|---|
| Google OAuth 2.0 | User authentication and Google Calendar connection | /auth/google/* and /google/* routers |
| Google Calendar API | Export Zemen events to user's Google Calendar | POST /google/export-event/{id} |
| Gmail SMTP (TLS) | Email reminder delivery | ReminderService daemon thread; smtplib with TLS |

# 7. Sprint Development Overview

Zemen was developed across four one-week sprints following the Scrum framework. Each sprint followed the standard Scrum cycle: Planning → Implementation → Testing → Review → Retrospective. This section documents every sprint in full detail.

## 7.1 Sprint 1 - Foundation and Authentication

### 1. Planning

#### Sprint Goal

Deliver a running three-tier application with user registration, login (email/password + Google OAuth), JWT-based authentication, profile management, and a correct database schema that will support the entire project without structural changes.

**User Story: Secure User Authentication**

| Field | Detail |
|---|---|
| As a | new user |
| I want to | register with my email and password and log in to receive a secure token |
| So that | my data is protected and only I can access my events and profile |
| Acceptance Criteria | Registration hashes password with bcrypt. Login returns JWT. Unauthenticated requests return 401. Token expires after 60 minutes. |
| Story Points | 5 |

**User Story: Google OAuth Login**

| Field | Detail |
|---|---|
| As a | user with a Google account |
| I want to | log in using Google OAuth 2.0 without creating a separate password |

| | |
|---|---|
| So that | I can access Zemen with a single click using my existing Google identity |
| Acceptance Criteria | Login URL endpoint redirects to Google. Callback creates/retrieves user. JWT is returned to frontend. |
| Story Points | 8 |

## 2. Implementation

### Sprint Backlog

| Item | Status | Owner |
|---|---|---|
| Design and create all six database tables | Done | Both |
| Set up Docker Compose (React + FastAPI + PostgreSQL) | Done | Aklesia |
| Implement /auth/register with bcrypt | Done | Meron |
| Implement /auth/login returning JWT | Done | Meron |
| Implement Google OAuth 2.0 login flow | Done | Aklesia |
| Implement /profile GET and PUT endpoints | Done | Both |
| Create Figma wireframes for all pages | Done | Meron |

### Major Achievements

- The full Docker Compose stack - React (port 5173), FastAPI (port 8000), PostgreSQL (port 5432) launched with a single command and passed health checks from day one.
- The database schema was finalized: users, events, event_participants, event_snapshots, holidays, reminder_logs. This schema held for the entire project with zero structural changes.
- Both authentication flows - email/password with bcrypt+JWT and Google OAuth 2.0 were implemented and tested.
- Figma wireframes were created for Calendar, Event Form, Convert, Settings, and Diff Viewer pages.

### Challenges and Impediments

- Google OAuth configuration required correctly setting redirect URIs in the Google Cloud Console. An initial mismatch caused 400 errors during callback. Resolved by aligning GOOGLE_REDIRECT_URI in docker-compose.yml with the registered URI.
- FastAPI's automatic 422 validation conflicted with our initial login endpoint design. Resolved in Sprint 3 (see Section 8.3).

## 3. Testing

Sprint 1 testing focused on manual verification via Postman. Authentication endpoints (register, login, wrong password, and unauthenticated access) were tested manually and formed the basis of the 01-Authentication folder in the final Postman collection.

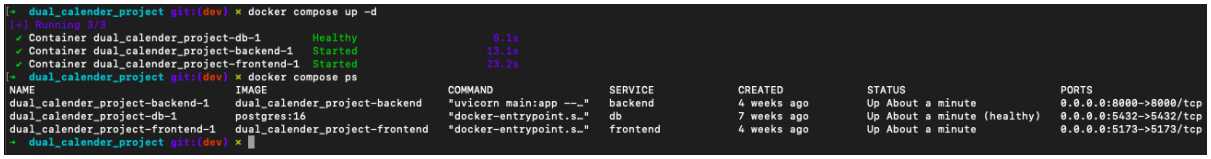


*Figure 2: Sprint 1 - Docker Compose stack running successfully*

## 4. Review

### Key Deliverables and Metrics

| Deliverable | Status | Notes |
|---|---|---|
| Docker Compose environment | Delivered | All three services pass health checks |
| Database schema (6 tables) | Delivered | No changes required in subsequent sprints |
| Email/password auth | Delivered | bcrypt + JWT; 60-minute expiry |
| Google OAuth 2.0 login | Delivered | Login and account creation flows working |
| Profile CRUD | Delivered | GET and PUT /profile endpoints |
| Figma wireframes | Delivered | All 5 pages wireframed |

## 5. Retrospective

**What Went Well**

- Docker Compose setup was smooth. Having a reproducible environment from the first sprint eliminated environment-related debugging for the rest of the project.
- Database schema design was thorough. Including event_snapshots and reminder_logs from the beginning avoided painful migrations later.

**What to Improve**

- Google OAuth setup took longer than estimated. Future OAuth integrations should be allocated more time.
- Figma wireframes were created after implementation started rather than before. In Sprint 2 we reversed this order.

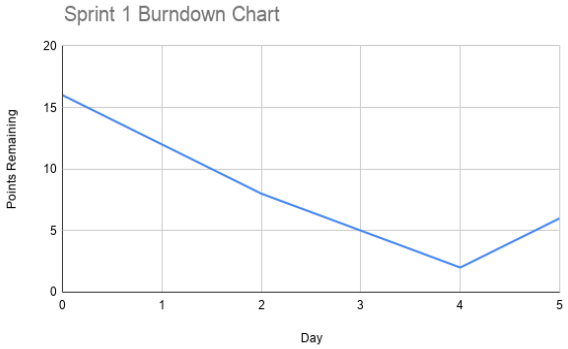


*Figure 3: Sprint 1 Burndown Chart*

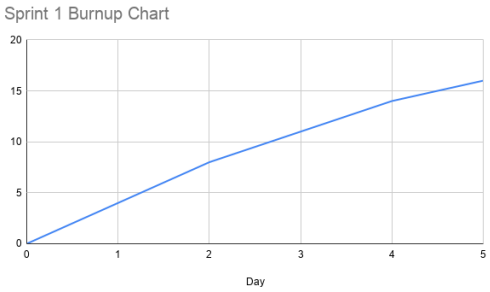


*Figure 4: Sprint 1 Burnup Chart*

## 7.2 Sprint 2 - Date Conversion Engine and Event CRUD

### 1. Planning

**Sprint Goal**

Deliver a correct, edge-case-tested Ethiopian–Gregorian conversion engine and a complete event management system with full CRUD, recurrence support, date-range filtering, and full-text search.

**User Story: Date Conversion**

| Field | Detail |
|---|---|
| As a | user working across both calendar systems |
| I want to | convert any date from Gregorian to Ethiopian or vice versa with correct handling of the 13th month and leap years |
| So that | I can schedule events and communicate dates accurately in either system |
| Acceptance Criteria | Pagume 6 is valid only in Ethiopian leap years. New Year falls on Sep 12 in non-Gregorian-leap years, Sep 11 in Gregorian leap years. Round-trip conversion is lossless. |
| Story Points | 8 |

**User Story: Event Management**

| Field | Detail |
|---|---|
| As a | registered user |
| I want to | create, view, update, and delete calendar events with support for Ethiopian or Gregorian dates, reminders, and recurrence |
| So that | I can manage my schedule across both calendar systems |
| Acceptance Criteria | All four HTTP verbs work. Events stored in UTC. Recurrence (daily/weekly) supported. Search and date-range filter work. Timezone conversion on output. |
| Story Points | 8 |

## 2. Implementation

**Completed Backlog Items**

| Item | Status | Owner |
|---|---|---|
| ethiopian_date library integration and wrapper | Done | Aklesia |
| DataConversionService with Pagume and New Year edge cases | Done | Aklesia |
| GET /convert/g2e and /convert/e2g endpoints | Done | Aklesia |
| POST /events - create with UTC storage | Done | Meron |
| GET /events - list with search and date filter | Done | Meron |
| GET /events/{id} - retrieve single event | Done | Meron |
| PUT /events/{id} - update with version field | Done | Both |
| DELETE /events/{id} - remove event | Done | Meron |
| Holiday seed data and /holidays endpoints | Done | Aklesia |
| React Convert page and EventForm with calendar toggle | Done | Both |

**Major Achievements**

- DataConversionService correctly handles all edge cases: Pagume 5 vs 6 days (Ethiopian leap year rule: year % 4 == 3), the New Year boundary (September 12 in non-Gregorian-leap years, September 11 in Gregorian leap years), and the 13-month Ethiopian year structure.
- Event CRUD system stores all times in UTC and converts to user timezone on output, ensuring time zone correctness regardless of where the user or server is located.
- The EventForm React page allows users to toggle between Gregorian (datetime-local input) and Ethiopian (year/month/day fields) date entry, with automatic conversion via the API before saving.

**Challenges and Impediments**

- Off-by-one errors in the New Year boundary calculation were caught through the round-trip parametrized Pytest tests added in Sprint 2. This validated the investment in test-first edge case thinking.
- Timezone handling initially double-converted some dates. Resolved by enforcing the rule: store UTC always, apply the timezone only on the way out to the client.

### 3. Testing

test_date_conversion_edge_cases.py was written this sprint: 16 tests covering New Year boundary dates, Pagume 5 vs 6 days, round-trip losslessness (6 parameterized date pairs), all 13 Ethiopian month names, and year-boundary cases.

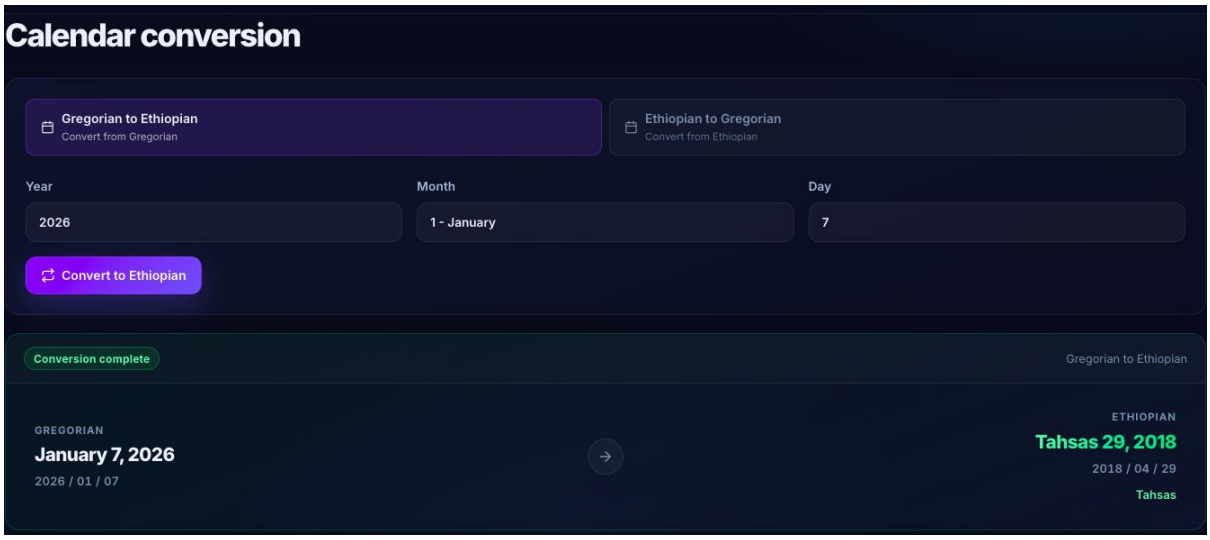


*Figure 5: Sprint 2 - Date Converter showing Gregorian → Ethiopian conversion*

### 4. Review

| Deliverable | Status | Notes |
|---|---|---|
| DataConversionService (full edge cases) | Delivered | 16 Pytest tests, all passing |
| GET /convert/g2e and /convert/e2g | Delivered | Included in Postman folder 03 |
| Event CRUD (all 4 verbs) | Delivered | UTC storage; version field introduced |
| Ethiopian date entry in EventForm | Delivered | Calendar toggle works end-to-end |
| Holiday data and endpoints | Delivered | Ethiopian and Gregorian holidays seeded |

### 5. Retrospective

#### What Went Well

- The decision to add parameterized round-trip tests immediately caught off-by-one errors that would have been very hard to debug later.
- The event version field was introduced in Sprint 2 even though optimistic locking was not fully implemented until Sprint 3 - this forward planning avoided a schema migration.

**What to Improve**

- The Convert page UI is functional but unstyled. Added to Sprint 4 polish backlog.

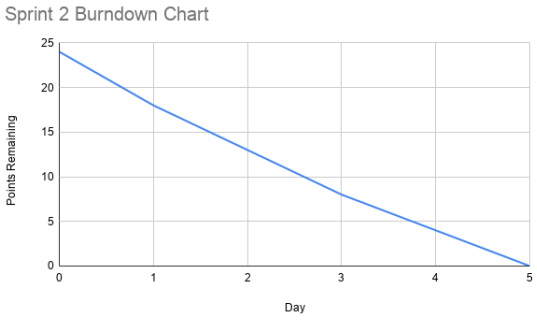


*Figure 6: Sprint 2 Burndown Chart*

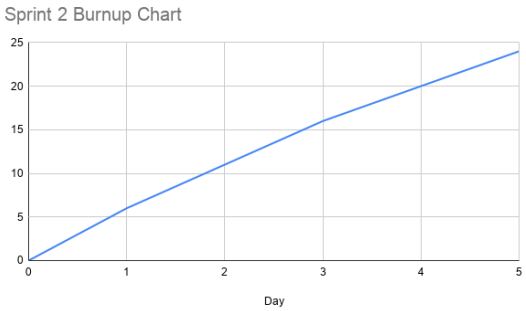


*Figure 7: Sprint 2 Burnup Chart*

## 7.3 Sprint 3 - Collaboration, Scheduling Engine, and Reminders

### 1. Planning

**Sprint Goal**

Deliver the full collaborative event system (role-based sharing, optimistic locking, version conflict detection, field-level diff), the multi-stage smart scheduling engine (interval merging, gap finding, constraint-based slot ranking), and the email reminder daemon.

**User Story: Collaborative Event Editing with Conflict Detection**

| Field | Detail |
|---|---|

| | |
|---|---|
| As a | shared event editor |
| I want to | be notified when my save would overwrite someone else's concurrent changes, and be shown exactly what changed |
| So that | no data is silently lost and I can make an informed decision about whether to force-save or discard my changes |
| Acceptance Criteria | Stale version returns 409 with VERSION_CONFLICT code. GET /events/{id}/diff returns field-level changes with from/to values. Frontend shows conflict message with options to review diff or force-save. |
| Story Points | 13 |

**User Story: Smart Scheduling**

| Field | Detail |
|---|---|
| As an | event organiser with multiple participants |
| I want to | get ranked suggestions for the best meeting time that avoids everyone's conflicts |
| So that | I can quickly schedule a meeting without manually checking each participant's calendar |
| Acceptance Criteria | GET /events/{id}/rank returns slots sorted by score. Required participant busy = slot excluded. Optional participant busy = +100 score. Outside work hours = +30 score. Earlier slots score better. |
| Story Points | 13 |

## 2. Implementation

### Completed Backlog Items

| Item | Status | Owner |
|---|---|---|
| POST /events/{id}/share - role assignment | Done | Meron |
| Role permission enforcement at all endpoints | Done | Meron |

| | | |
|---|---|---|
| Optimistic locking - version check on PUT | Done | Aklesia |
| EventSnapshot model and write-on-update logic | Done | Aklesia |
| DiffService - field-level comparison with structured change digest | Done | Aklesia |
| GET /events/{id}/diff endpoint | Done | Aklesia |
| IntervalService - normalize, merge, find gaps | Done | Both |
| RankingService - candidate generation + penalty scoring | Done | Both |
| GET /events/{id}/rank endpoint | Done | Both |
| ReminderService - daemon thread + duplicate prevention | Done | Meron |
| EventDiff React page | Done | Aklesia |
| Ranked slots panel in EventForm | Done | Both |

**Major Achievements**

- The optimistic locking implementation is production-grade: every update atomically checks and increments the version field, and the 409 response includes enough information (current_version, your_version) for the client to offer a meaningful recovery path.
- The DiffService compares snapshots field by field and produces a plain-text digest listing the changed field names (e.g. “Changes: title, reminder_minutes”). providing a concise, auditable record of each change set, analogous to the change history in Google Docs.
- The scheduling pipeline (IntervalService → merge → gaps → slots → RankingService) correctly handles: participants with no events (zero busy intervals), fully blocked windows (empty result), gaps shorter than the requested duration (skipped), and holidays as all-day busy blocks.
- The ReminderService runs as a daemon thread, polls every 10 seconds, and uses the reminder_logs table to guarantee at-most-once delivery even across server restarts.

**Challenges and Impediments**

- Merging overlapping and touching intervals (e.g., (09:00, 10:30) and (10:30, 11:00)) required careful boundary handling. The merge function was refined after test failures revealed a touching-but-non-overlapping edge case.
- Holiday integration into the scheduling engine required extending busy_intervals_for_user to call holiday_intervals and prepend them before merge_intervals. This was an unplanned but necessary design change identified during Sprint 3 testing.

### 3. Testing

test_scheduling_pipeline.py was written this sprint: 15 tests covering two-user conflict scenarios, fully-blocked windows, required vs optional participant handling, sort-order guarantees, role permission checks, and fragmented interval merging.

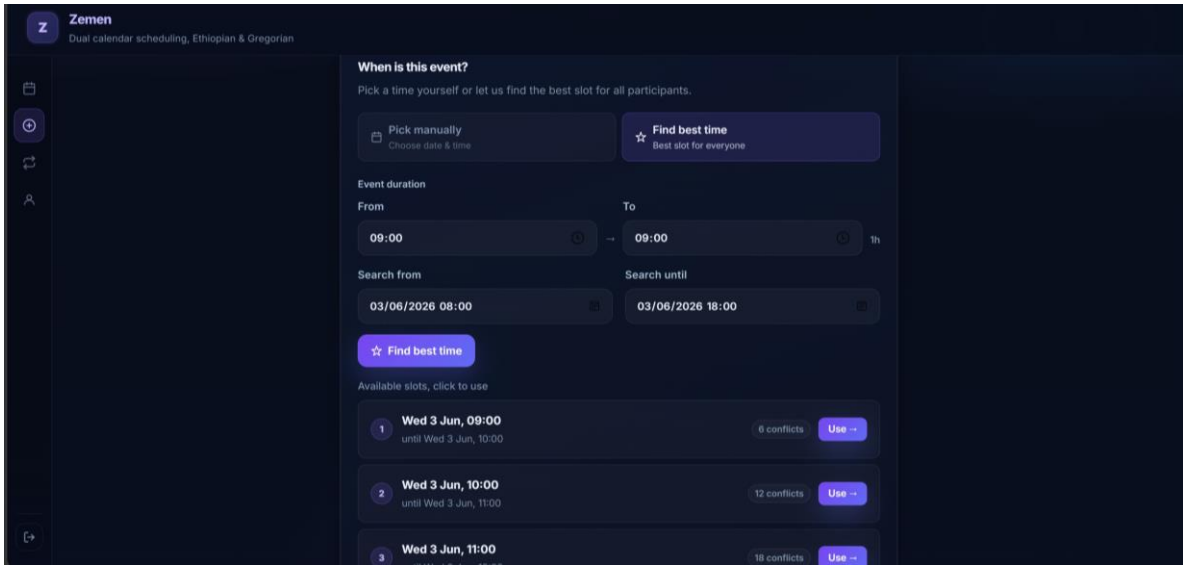


*Figure 8: Sprint 3 - Ranked time slot suggestions in the Event Form*

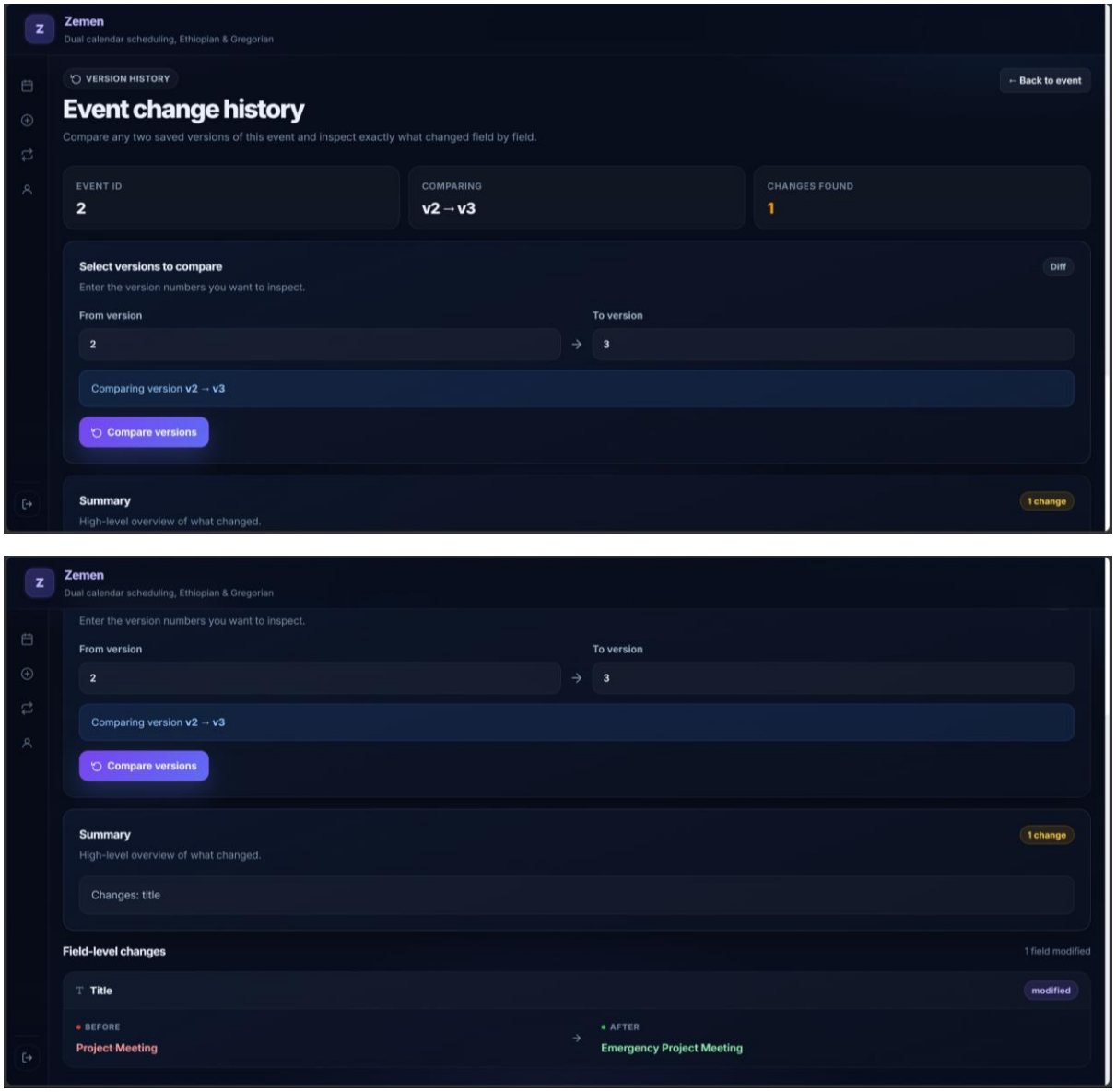


*Figure 9: Sprint 3 - Event Diff Viewer showing field-level change history*

## 4. Review

| Deliverable | Status | Notes |
|---|---|---|
| Role-based event sharing | Delivered | owner/editor/viewer enforced at all endpoints |
| Optimistic locking + 409 Conflict | Delivered | VERSION_CONFLICT with current_version in response |
| Field-level diff (DiffService) | Delivered | Structured plain-text change digest included |

| | | |
|---|---|---|
| IntervalService (merge + gaps) | Delivered | Handles all edge cases |
| RankingService (candidates + scoring) | Delivered | Penalty table: required=exclude, optional=+100, off-hours=+30 |
| ReminderService (daemon thread) | Delivered | At-most-once delivery via reminder_logs |
| EventDiff React page | Delivered | Accessible from calendar and conflict dialog |

## 5. Retrospective

### What Went Well

- Sprint 3 was the most technically demanding sprint and everything was delivered. The algorithmic work on the scheduling pipeline, particularly the penalty-based ranking, produced results that feel genuinely useful, not just technically correct.
- The test suite grew substantially this sprint. Having tests for the scheduling pipeline made refactoring the holiday integration straightforward.

### What to Improve

- The EventDiff page UI is minimal. Added to Sprint 4 polish list.
- The ranked slots panel in the EventForm only shows slot times and scores, not participant-level detail. A future improvement would show which participants are busy for each slot.

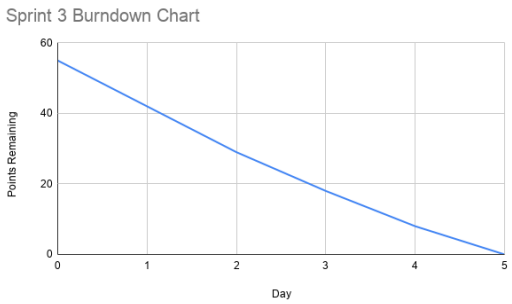


*Figure 10: Sprint 3 Burndown Chart*

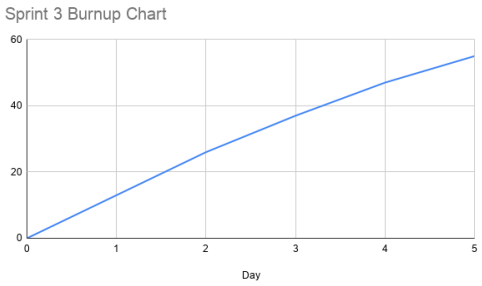


*Figure 11: Sprint 3 Burnup Chart*

## 7.4 Sprint 4 - Testing, UI Polish, and Docker Deployment

### 1. Planning

#### Sprint Goal

Reach comprehensive test coverage (116 Pytest + 93 Postman), polish the UI across all pages, finalize Docker Compose deployment, generate all UML diagrams, and write the project report.

#### User Story: Comprehensive Test Coverage

| Field | Detail |
|---|---|
| As the | project team |
| We want to | achieve full coverage of all algorithms and all API endpoints with automated tests |
| So that | we can confidently demonstrate the system's correctness and reliability |
| Acceptance Criteria | 116 Pytest tests all pass. 93 Postman tests all pass. Postman collection is rerunnable (timestamp email, live version fetch). |
| Story Points | 8 |

### 2. Implementation

#### Completed Backlog Items

| Item | Status | Owner |
|---|---|---|
| Extend Pytest suite to 116 tests | Done | Both |

| | | |
|---|---|---|
| Build Postman collection (93 tests, 11 folders) | Done | Both |
| Add Date.now() timestamp to Postman registration for rerunnability | Done | Meron |
| Add pre-request script for live version fetch in Postman | Done | Meron |
| UI polish: Calendar, EventForm, Convert, Settings pages | Done | Aklesia |
| Generate 6 PlantUML diagrams and 2 use case diagrams | Done | Both |
| Verify Docker Compose full build from clean state | Done | Aklesia |
| Write project report | Done | Both |

**Major Achievements**

- The Postman collection was made fully rerunnable: the registration request uses Date.now() in the email address to avoid duplicate registration errors, and the Update Event request uses a pm.sendRequest pre-request script to fetch the current version before every run.
- The FastAPI login header issue (credentials sent as JSON body vs query parameters) was identified and corrected. The login endpoint takes email and password as query parameters, not a JSON body. This was the root cause of a cascade failure in the early Postman collection.
- All 116 Pytest tests pass. All 93 Postman tests pass. Average API response time: 106ms.

**Challenges and Impediments**

- The Postman version-staleness problem: the initial Update Event request sent a hardcoded version number that became stale after the first run. Solved with the pre-request script.
- The FastAPI header validation issue: the login endpoint was sending a JSON body to a query-parameter endpoint, causing 422 errors and silent token failures in all downstream tests. Required careful debugging of the response chain.

## 3. Testing


*Figure 12: Sprint 4 - Full Pytest run showing 116 tests passing*

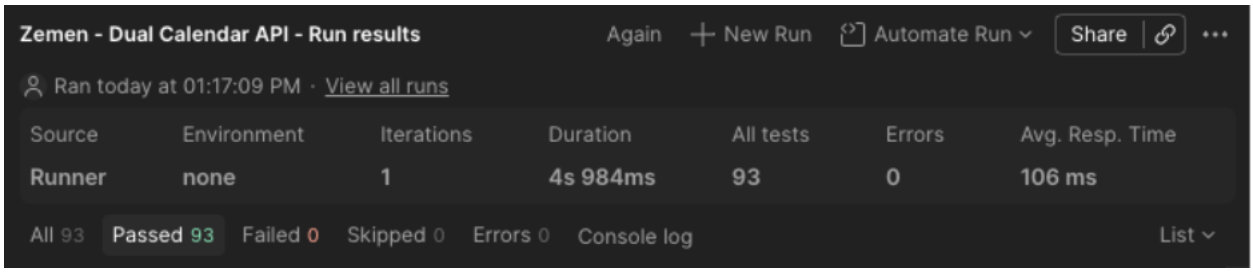


*Figure 13: Sprint 4 - Postman Collection Runner showing all 93 tests passing*

## 4. Review

| Deliverable | Status | Notes |
|---|---|---|
| 116 Pytest tests | Delivered | All passing; 0 failures; 7.43s runtime |
| 93 Postman tests (11 folders) | Delivered | All passing; avg 106ms response time; fully rerunnable |
| UI polish across all pages | Delivered | Calendar, EventForm, Convert, Settings, Diff Viewer |
| 6 UML diagrams (PlantUML) | Delivered | Class, 4× Sequence, Component |
| Docker Compose clean build | Delivered | Full rebuild from scratch verified |
| Project report | Delivered | This document |

## 5. Retrospective

### What Went Well

- The rerunnable Postman collection is a genuine quality artifact - it can be run against any deployed instance of Zemen and will produce a full pass/fail report without any manual setup.
- The decision to introduce the version field in Sprint 2 and build the snapshot mechanism in Sprint 3 meant there was nothing to retrofit in Sprint 4.

**What to Improve**

- Report writing took longer than expected and should be allocated its own sprint backlog item with a story point estimate from the beginning.
- The UI could benefit from a proper design system (Tailwind CSS or a component library). Currently styled with inline CSS which is functional but not scalable.

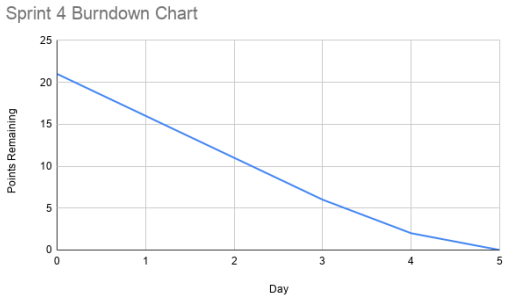


*Figure 14: Sprint 4 Burndown Chart*

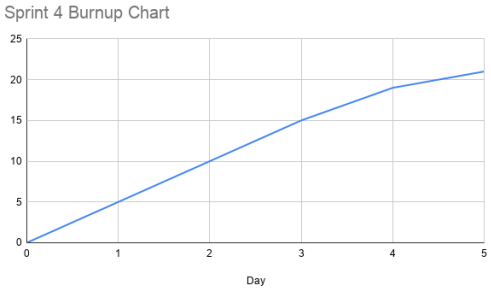


*Figure 15: Sprint 4 Burnup Chart*

# 8. System Architecture

## 8.1 Three-Tier Web Architecture

Zemen follows a strict three-tier architecture: Presentation (React frontend), Application (FastAPI backend), and Data (PostgreSQL database). This separation of concerns means each tier can be developed, tested, and scaled independently.

| Tier | Technology | Port | Responsibility |
|---|---|---|---|
| Presentation | React 19 + Vite | 5173 | Renders the UI; handles routing; stores JWT in localStorage; attaches Bearer token to every authenticated request via authHeaders() |

| | | | |
|---|---|---|---|
| Application | FastAPI (Python 3.12) | 8000 | All business logic; 8 focused routers; 6 service classes; JWT validation; role enforcement; external service calls |
| Data | PostgreSQL 16 | 5432 | Persistent storage for all application data; 6 tables; ACID-compliant transactions for optimistic locking |

### Frontend Routing

The React frontend uses React Router v7 with a RequireAuth wrapper component that redirects unauthenticated users to /login. The authenticated routes are:

| Route | Component | Purpose |
|---|---|---|
| / | Calendar | Month-view calendar grid with event display, create, edit, delete |
| /convert | Convert | Date conversion in both directions with result display |
| /settings | Settings | Profile viewing and editing (preferred calendar, timezone, language) |
| /events/new | EventForm | Create a new event (Gregorian or Ethiopian date input) |
| /events/:id/edit | EventForm | Edit existing event with version display and ranked slot suggestions |
| /events/:id/diff | EventDiff | View field-level diff between any two event versions |

### Backend API Structure

The FastAPI backend is organized into 8 routers, each handling its own domain, with business logic extracted into service modules:

| Router | Prefix | Key Endpoints | Service |
|---|---|---|---|

| | | | |
|---|---|---|---|
| auth | /auth | register, login, google/login-url, google/callback | SecurityService |
| profile | /profile | GET, PUT /profile | — |
| events | /events | CRUD, /share, /participants, /diff, /conflicts, /suggest, /rank | IntervalService, RankingService, DiffService |
| conversion | /convert | g2e, e2g | DataConversionService |
| scheduling | (events) | /suggest, /rank | IntervalService, RankingService |
| diff | (events) | /diff | DiffService |
| holidays | /holidays | GET, POST, DELETE | — |
| google | /google | status, connect-url, export-event/{id} | — |

## 8.2 Database Design

The data model was designed in Sprint 1 and required zero structural changes throughout the project, a testament to the upfront schema planning.

| Table | Purpose | Key Fields |
|---|---|---|
| users | User accounts, preferences, OAuth tokens | id, email, hashed_password, preferred_calendar, timezone, google_access_token, google_refresh_token |
| events | Core event data stored in UTC | id, user_id (FK), title, description, start_time_utc, end_time_utc, timezone, version |
| event_participants | Role-based sharing junction table | id, event_id (FK), user_id (FK), role (owner\|editor\|viewer) |

| | | |
|---|---|---|
| event_snapshots | Immutable audit trail - one row per version | id, event_id (FK), version, title, description, start_time_utc, end_time_utc, timezone, reminder_minutes |
| holidays | Ethiopian and Gregorian public holidays | id, name, eth_year/month/day, resolved_date, calendar_type |
| reminder_logs | Sent reminder records for duplicate prevention | id, event_id (FK), recipient_email, sent_at |

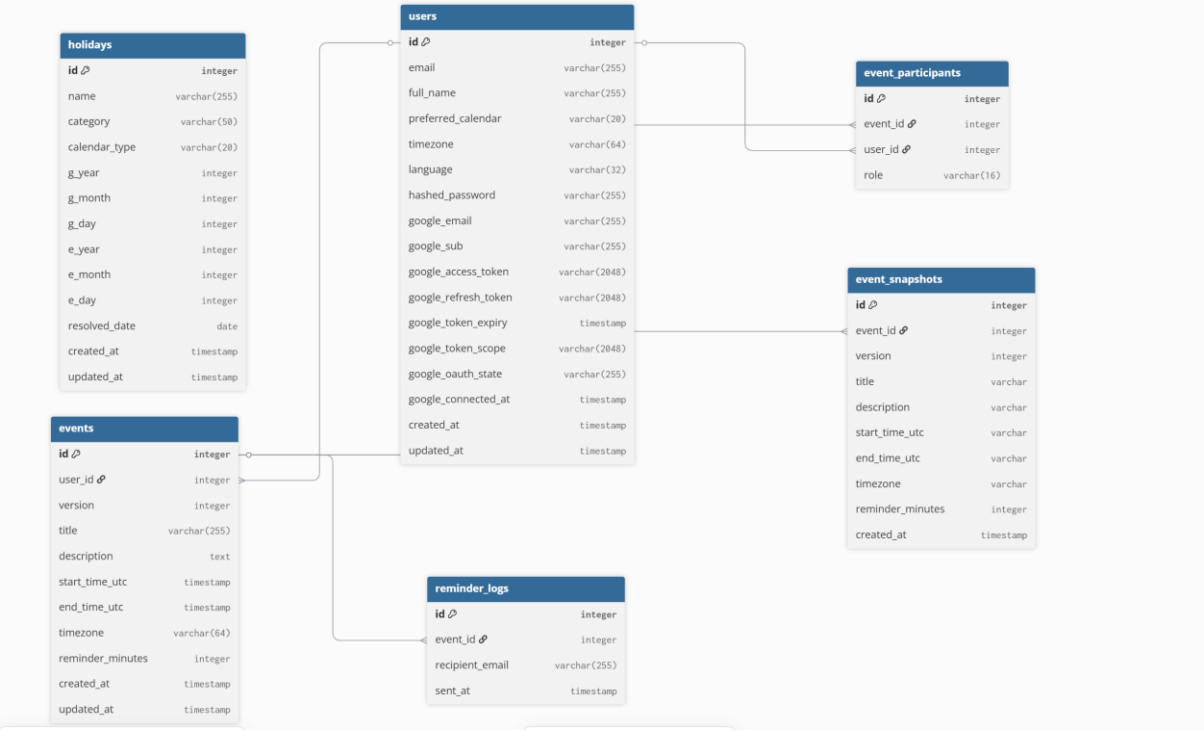


*Figure 16: Zemen Database Entity-Relationship Diagram*

## 8.3 Deployment Architecture

The entire system is containerized using Docker Compose. Each tier runs in its own container, connected via Docker's internal network. The database uses a named volume (pgdata) for data persistence across container restarts.

| Service | Image | Port | Health Check | Depends On |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| db | postgres:16 | 5432 | pg_isready -U dualcal | — |
| backend | Built from ./backend Dockerfile | 8000 | — | db (condition: service_healthy) |
| frontend | Built from ./frontend Dockerfile | 5173 | — | backend |

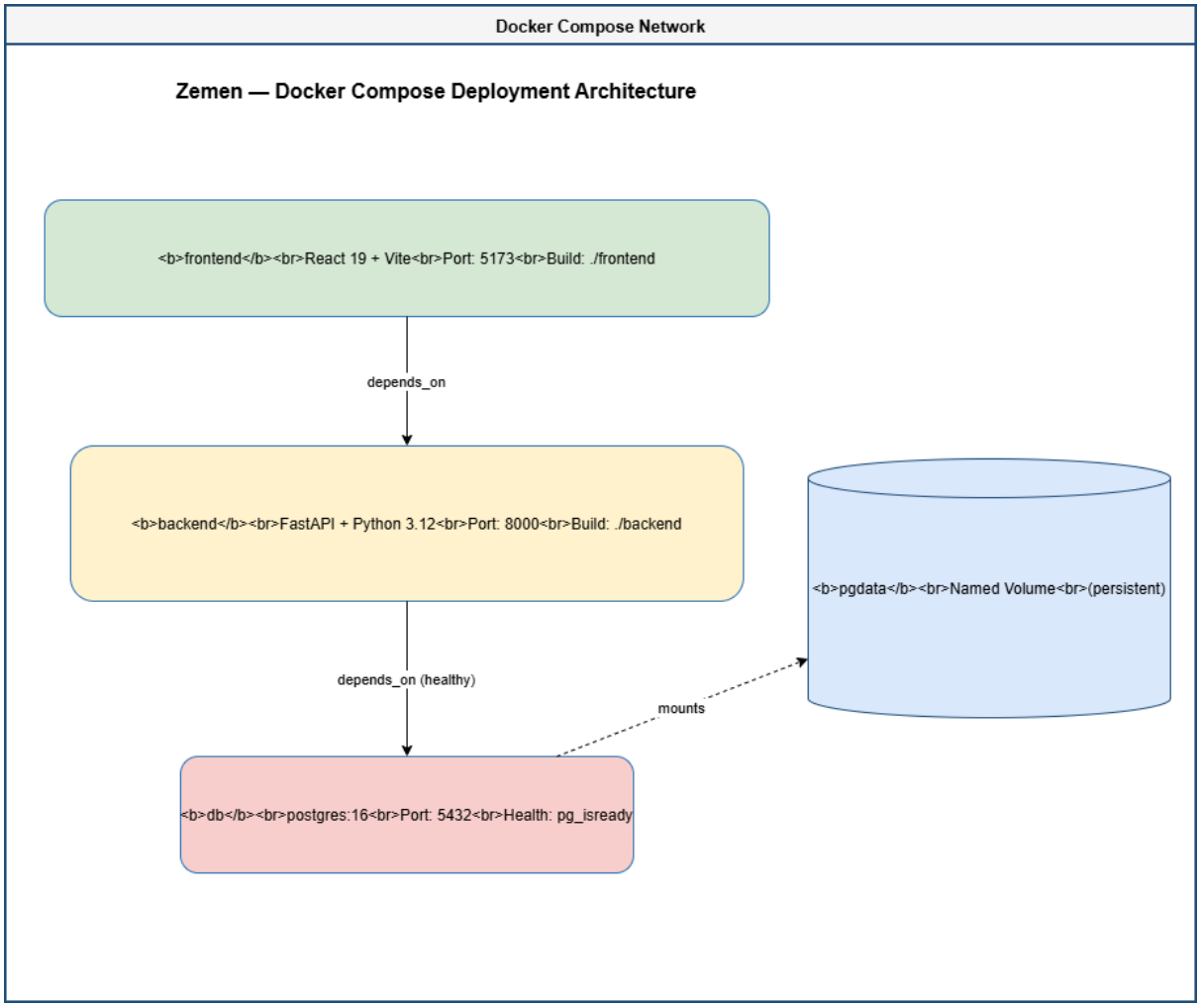


*Figure 17: Zemen Deployment Architecture - Docker Compose*

# 9. UML Diagrams

Six UML diagrams were produced for Zemen using PlantUML. Each is documented below with a description of what it shows and the design decisions it captures.

## 9.1 Class Diagram

The class diagram shows all six database models with their fields and types, the relationships between them, and the six core service classes with their method signatures.

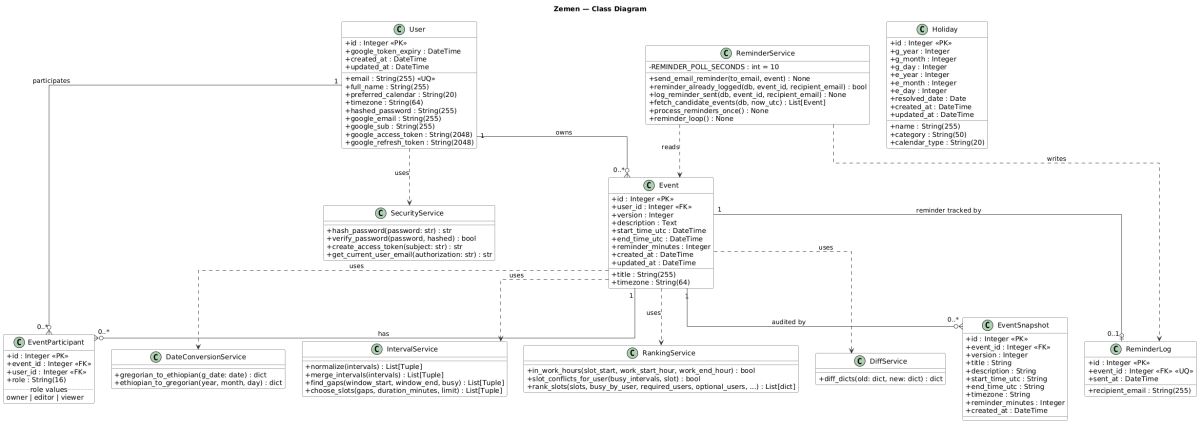


*Figure 18: Zemen Full Class Diagram*

## 9.2 Sequence Diagram: Authentication

Shows the full register, login,  JWT, authenticated request flow, including the Google OAuth 2.0 path and the alt blocks for invalid credentials and expired tokens.

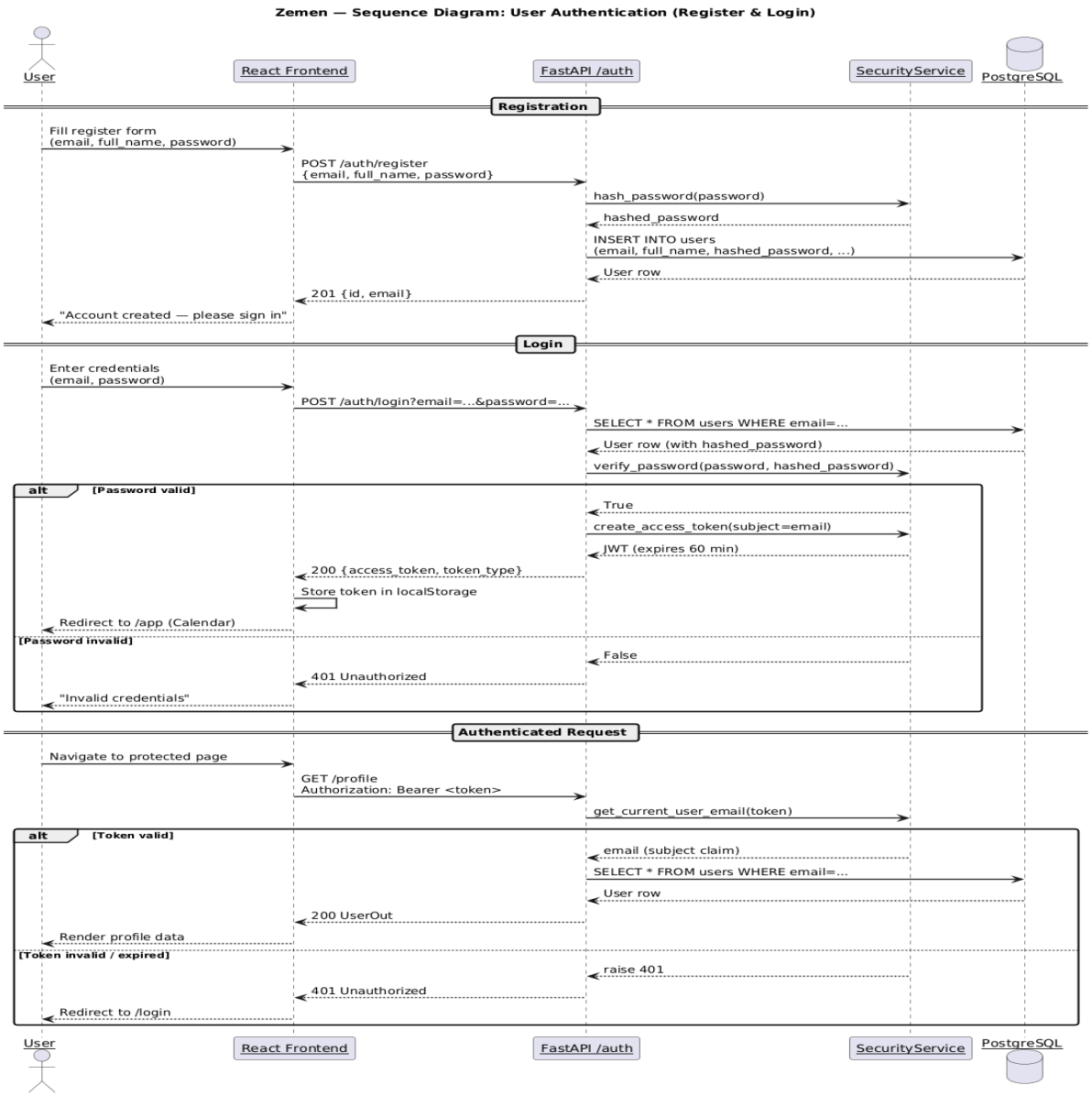


*Figure 19: Authentication Sequence Diagram*

## 9.3 Sequence Diagram: Create and Share Event

Documents the event creation flow (POST /events → snapshot write), participant sharing (POST /events/{id}/share), and how a shared event appears on a participant's calendar.

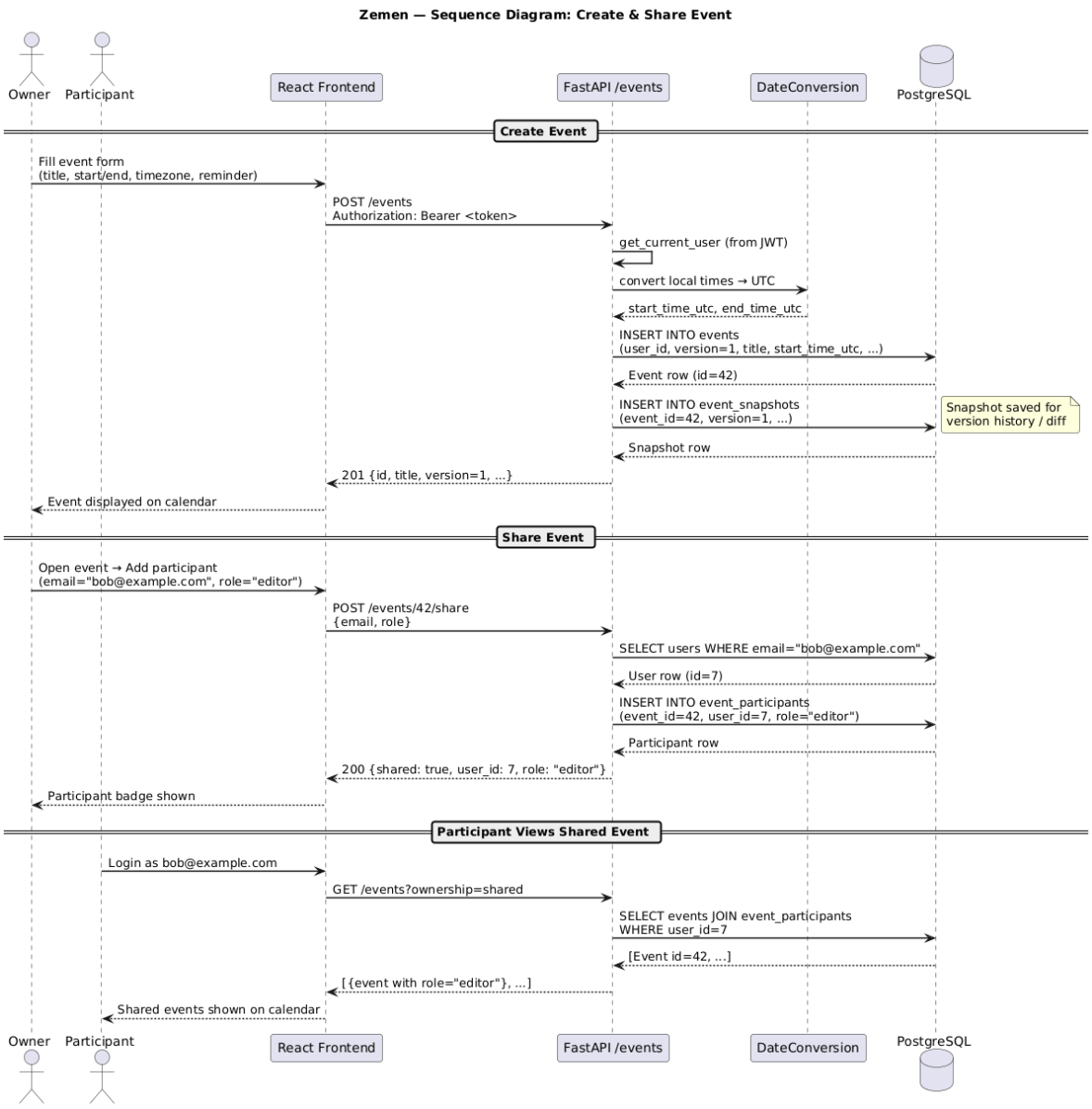


*Figure 20: Create and Share Event Sequence Diagram*

## 9.4 Sequence Diagram: Optimistic Locking and Version Conflict

Shows two editors loading the same event simultaneously. Editor A saves successfully. Editor B receives a 409 Conflict, reviews the field-level diff, and chooses to force-save.

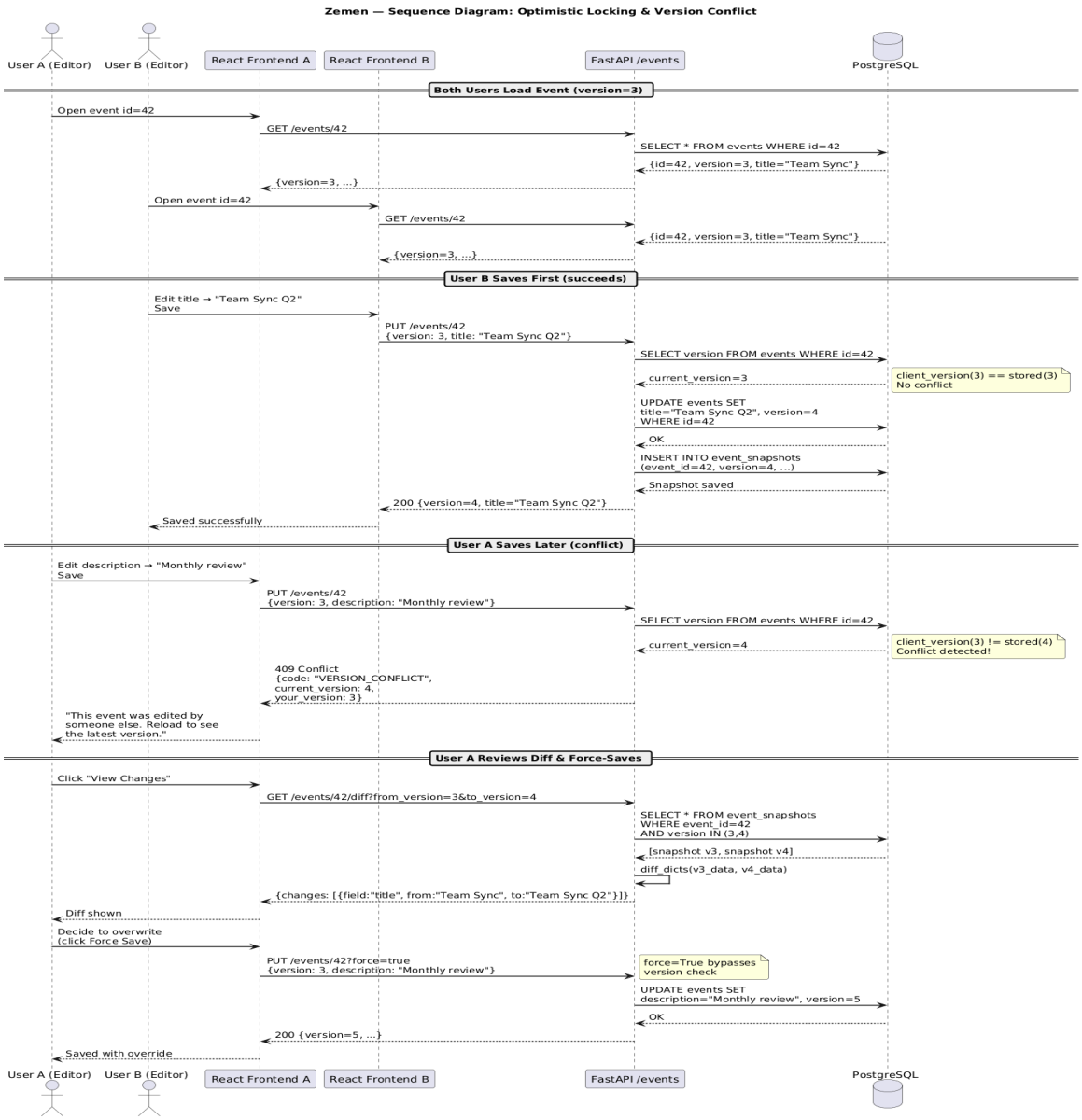


*Figure 21: Optimistic Locking - Version Conflict Resolution Sequence Diagram*

## 9.5 Sequence Diagram: Smart Scheduling Pipeline

Documents the full scheduling pipeline from the organiser opening the ranked slot panel, through participant busy-time collection, interval merging, gap finding, candidate generation, and constraint-based ranking, to the result displayed in the UI.

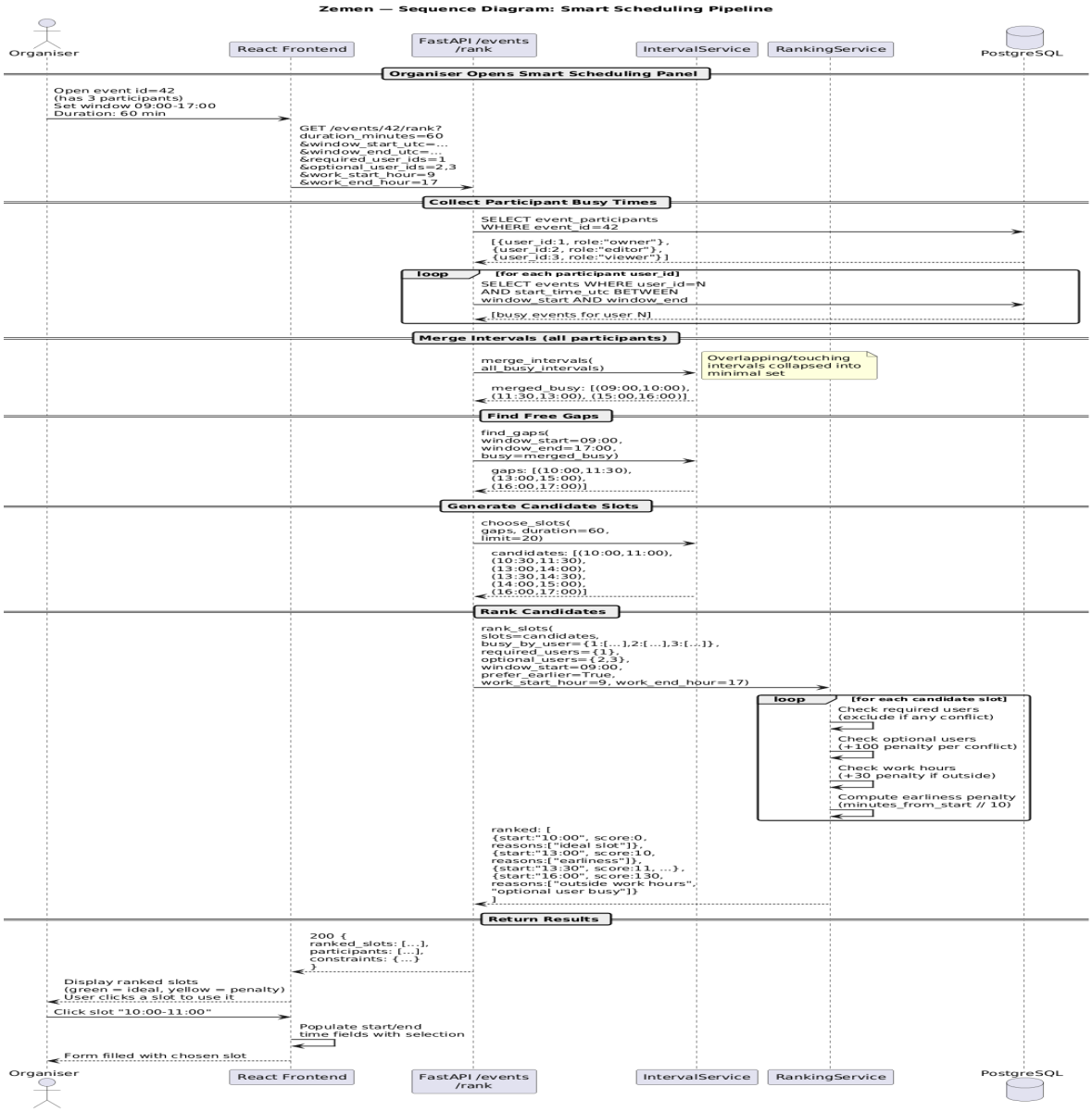


*Figure 22: Smart Scheduling Pipeline Sequence Diagram*

## 9.6 Component / Architecture Diagram

Shows the full system component view: the React frontend with its six pages, the FastAPI backend with its eight routers and six service classes, the PostgreSQL ORM models, and the external service integrations.

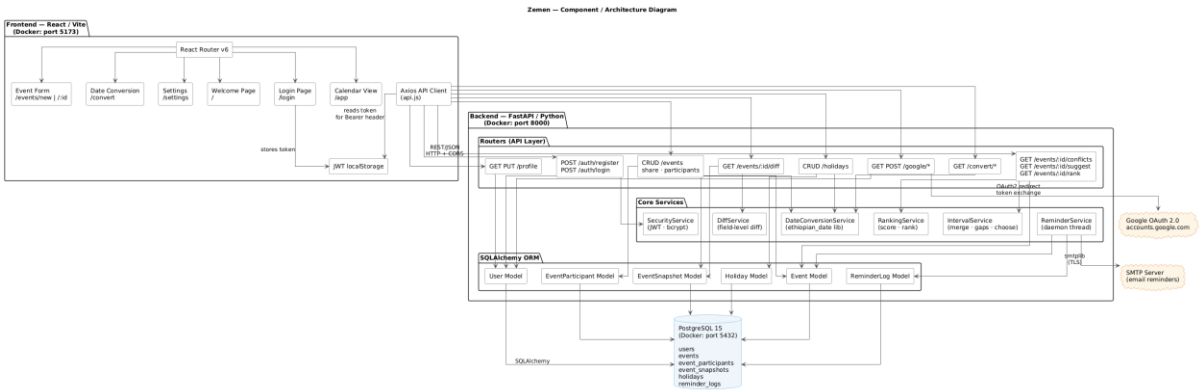


*Figure 23: Zemen System Component Diagram*

## 9.7 Use Case Diagrams

### Registered User Use Cases

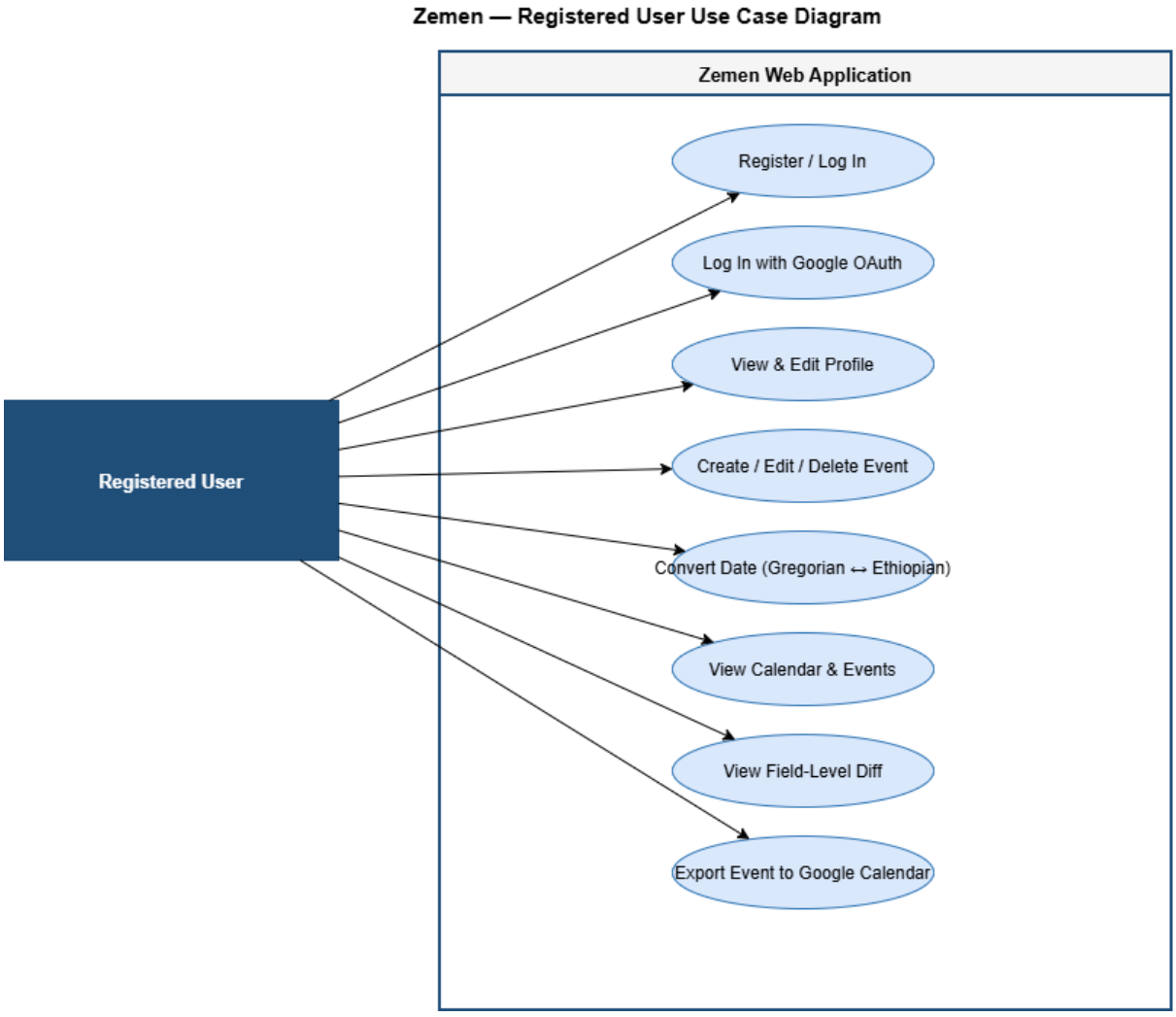


*Figure 24: Registered User Use Case Diagram*

**Admin / Event Owner Use Cases**

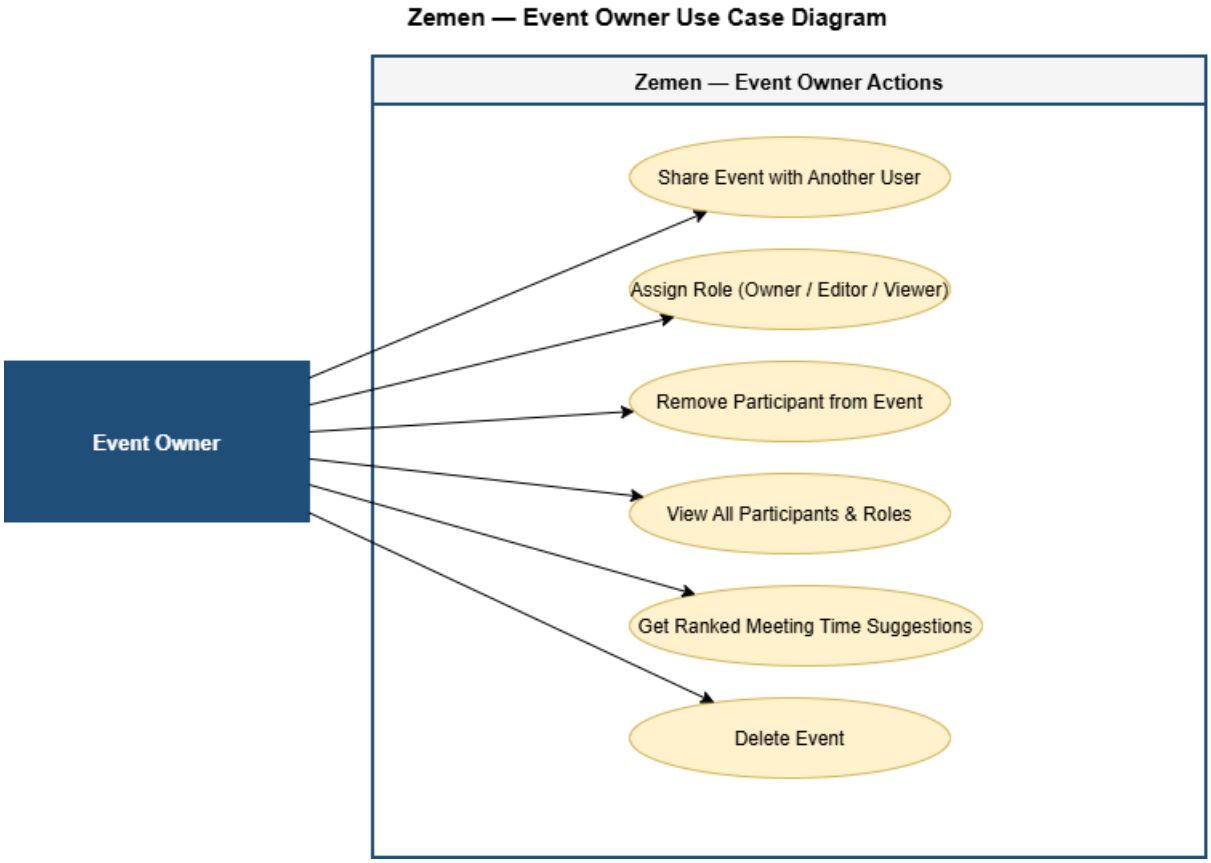


*Figure 25: Event Owner Use Case Diagram*

# 10. Feature Implementation

## 10.1 Ethiopian–Gregorian Date Conversion Engine

The conversion engine is built on top of the ethiopian_date Python library, extended by our DataConversionService wrapper to handle the edge cases the base library does not address.

| Edge Case | Rule | Implementation |
|---|---|---|
| Ethiopian leap year | Ethiopian year % 4 == 3 is a leap year | Pagume has 6 days; otherwise 5 |
| New Year in non-Gregorian-leap year | Ethiopian New Year = September 12 | Handled in g2e boundary check |

| | | |
|---|---|---|
| New Year in Gregorian leap year | Ethiopian New Year = September 11 | Detected via Gregorian leap year rule |
| 13th month (Pagume) | 5 or 6 days depending on leap year | Validated before conversion; invalid Pagume 6 in non-leap year rejected with 422 |

## 10.2 Event Management with Optimistic Locking

Every event carries a version integer, starting at 1 and incrementing with each successful update. The update flow is:

1. Client loads event and receives current version (e.g., version=3).
2. Client sends update with { version: 3, title: 'New title', ... }.
3. Backend compares submitted version against database version.
4. If they match: update proceeds, version becomes 4, snapshot is written.
5. If they differ: 409 Conflict returned with current_version, your_version, code: 'VERSION_CONFLICT'.

The frontend's EventForm shows the current version and handles the 409 response by displaying a conflict message with options to reload (get latest) or view the diff.

## 10.3 Field-Level Diff and Version History

Every successful event update writes an EventSnapshot, an immutable record of the event state at that version. The DiffService retrieves two snapshots and compares them field by field:

| Field | From | To |
|---|---|---|
| title | Team Sync | Team Sync Q2 |
| reminder_minutes | 30 | 60 |

A plain-text digest listing the changed field names is included in the response (e.g. “Changes: title, reminder_minutes”), providing a concise, human-readable summary of each change set.

## 10.4 Scheduling Algorithm: 5-Stage Pipeline

The smart scheduling engine runs a five-stage pipeline when GET /events/{id}/rank is called:

| Stage | Function | Description |
|---|---|---|
| 1 | Collect busy intervals | For each participant, query all events in the search window. Each event → (start, end) tuple. Holidays → all-day (window_start, window_end) tuples. |
| 2 | merge_intervals() | Sort all intervals by start. Sweep through, merging overlapping or touching intervals into a minimal non-overlapping set. |
| 3 | find_gaps() | Given merged busy set and window boundaries, identify all free intervals where a meeting could fit. |
| 4 | choose_slots() | Walk each gap; generate all slots of the requested duration, advancing by the full meeting duration at each step. |
| 5 | rank_slots() | Score each candidate slot using the penalty table. Sort by score ascending. Return top N results. |

**Penalty Scoring Table**

| Condition | Penalty |
|---|---|
| Required participant is busy at this slot | Slot excluded entirely (infinite penalty) |
| Optional participant is busy at this slot | +100 points per conflict |
| Slot is outside configured work hours | +30 points |
| Earliness preference | +N points where N = minutes from window start ÷ 10 |

**10.5 Email Reminder System**

The ReminderService runs as a Python daemon thread that starts when the FastAPI application boots. Every 10 seconds it:

1. Queries all events with reminder_minutes set.
2. if the event's reminder window is within the current polling interval.
3. Checks the reminder_logs table to see if this event's reminder has already been sent.
4. If not: sends the email via smtplib with TLS and writes a log entry.

The daemon=True flag ensures the thread exits automatically when the main process exits, without requiring explicit cleanup.

## 10.6 Google OAuth 2.0 and Calendar Export

Two separate OAuth flows are implemented:

- Login flow (/auth/google/*): Exchanges authorization code for an ID token; extracts user email; creates or retrieves the user account; returns a Zemen JWT.
- Calendar connection flow (/google/*): Exchanges code for access and refresh tokens; stores them in the users table; enables POST /google/export-event/{id} to push events to Google Calendar.

# 11. Testing

Zemen has a comprehensive two-tier testing strategy: Pytest for unit and integration testing of algorithms and backend logic, and Postman for end-to-end API integration testing. Together they provide 209 automated test executions, all passing.

## 11.1 Unit Tests -- Pytest (116 Tests)

| Test File | Tests | Coverage Area |
|---|---|---|
| test_date_conversion_edge_cases.py | 16 | Ethiopian–Gregorian conversion engine: New Year boundary (Sep 11 vs 12), Pagume leap year (5 vs 6 days), 6 round-trip pairs, all 13 month names, year-boundary dates |
| test_scheduling_pipeline.py | 15 | Full scheduling algorithm: two-user conflict, fully-blocked window, fragmented interval merging, required vs optional participant handling, sort order, version conflict logic, role permission checks, gap-shorter-than-duration |
| Additional test coverage | 85 | Authentication, profile, CRUD operations, holiday management, diff computation, reminder deduplication |


*Figure 26: Pytest -- 116 tests passing with zero failures*

## 11.2 Integration Tests - Postman (93 Tests)

The Postman collection covers all 11 endpoint folders in a top-to-bottom workflow that mirrors real user interactions. It consists of 28 requests containing a total of 93 pm.test() assertions, all passing. The table below shows the number of requests per folder; the Collection Runner reports the full assertion count of 93. The collection is designed to be fully rerunnable without any manual setup between runs.

| Folder | Requests | What Is Tested |
|---|---|---|
| 00 - Health | 1 | Server availability (GET /health) |
| 01 - Authentication | 4 | Register (timestamp email), valid login, wrong password (401), no-token access (401/422) |
| 02 - Profile | 2 | Get profile, update profile |
| 03 - Date Conversion | 3 | g2e (Jan 7 2026 → Tahsas 29 2018), e2g (Meskerem 1 2017), missing param (422) |
| 04 - Events CRUD | 6 | Create, list, get by ID, update (live version fetch), version conflict (409), non-existent (404) |
| 05 - Sharing & Participants | 2 | Share event, list participants |
| 06 - Scheduling | 3 | Conflict detection, slot suggestion, ranked slot results |
| 07 - Event Diff | 1 | Field-level diff between versions 1 and 2 |
| 08 - Holidays | 3 | List, create, delete |
| 09 - Google Calendar | 2 | Connection status, connect URL |
| 10 - Cleanup | 1 | Delete test event |

**Rerunnability Design**

- Registration: uses Date.now() in the email address (e.g. test1748300000000@zemen.test) so each run registers a fresh unique user without hitting duplicate email errors.
- Update Event: uses a pm.sendRequest pre-request script that fetches GET /events/{{event_id}} before the PUT, extracts the current version, and stores it as a collection variable. This ensures the version is always fresh regardless of how many updates have been run.

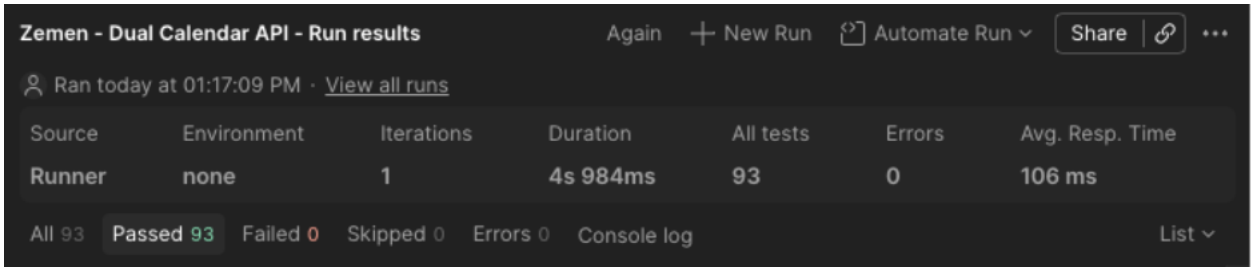


*Figure 27: Postman Collection Runner - 93 tests, all passing*

# 12. Challenges and How We Solved Them

## 12.1 Ethiopian Calendar Edge Cases

The conversion engine required careful handling of three distinct boundary conditions: the New Year date (September 11 vs 12 depending on the Gregorian leap year), the Pagume month length (5 vs 6 days depending on the Ethiopian leap year rule: year % 4 == 3), and round-trip losslessness. We caught several off-by-one errors through the 16 parametrised Pytest tests errors that would have been invisible in manual testing.

## 12.2 Optimistic Locking and Stale Versions in Postman

The initial Postman Update Event request sent a hardcoded version number (version: 1) that became stale after the first run. This caused all subsequent runs to receive a 409 Conflict. We solved it by adding a pre-request script: pm.sendRequest fetches the current event before the update, extracts the live version, stores it as a collection variable, and the update body uses that value.

## 12.3 FastAPI Header Validation Cascade Failure

The login endpoint in FastAPI takes email and password as bare function arguments, which FastAPI maps to query parameters. Our initial Postman collection sent a JSON body, receiving a 422 Unprocessable Entity in response. This silently prevented the JWT from being saved, causing every subsequent authenticated request to fail with 401. Once we corrected the login request to send credentials as query parameters, the entire collection passed.

## 12.4 Interval Merging: Touching Boundaries

The first version of merge_intervals treated touching intervals (e.g., (09:00, 10:30) and (10:30, 11:00)) as non-overlapping and left them separate. This produced a spurious 0-minute free gap between them. The fix was to change the merge condition from strictly less than to less than or equal to when comparing the end of the current merged interval with the start of the next.

## 12.5 Holiday Integration in the Scheduling Engine

The scheduling engine initially computed busy intervals only from events. When we added holiday support, we needed to inject all-day holiday blocks into the busy set before merging. This required extending busy_intervals_for_user to call a holiday_intervals helper and prepend the results before passing them to merge_intervals. This was an unplanned design change but straightforward to implement because IntervalService was already modular.

## 12.6 Google OAuth Redirect URI Mismatch

Google OAuth requires exact redirect URI matching between the Google Cloud Console registration and the application configuration. An initial mismatch between GOOGLE_REDIRECT_URI in docker-compose.yml and the registered URI caused 400 errors during the OAuth callback. Resolved by aligning both to http://localhost:8000/auth/google/callback.

## 12.7 Security Considerations for Future Production Deployment

For a university project, the security baseline is solid. However, for a production deployment, three improvements would be required:

- The login endpoint passes credentials as query parameters, which appear in server access logs. These should move to the request body (form data).
- The default SECRET_KEY fallback in the environment should be removed so the application refuses to start without an explicit environment variable.
- Google OAuth tokens stored in the users table should be encrypted at rest rather than stored as plaintext strings.

## 13. Conclusion

Zemen began as a straightforward dual-calendar event management application and evolved through the iterative Scrum process and the professor's challenge to exceed CRUD functionality into a technically substantial collaborative scheduling platform.

The three features that most distinguish Zemen from a simple CRUD application are:

- Optimistic locking with version conflict detection, a production-grade pattern that prevents silent data loss in collaborative editing, implemented correctly with atomic version increments, snapshot writing, and a meaningful client-facing recovery flow.
- The five-stage scheduling pipeline, nterval merging, gap finding, candidate generation, and penalty-based constraint ranking, which produces genuinely useful meeting time suggestions that are aware of Ethiopian holidays, participant roles, and work-hour preferences.
- The immutable event snapshot system with field-level diff, which provides the same kind of auditable change history found in professional tools like Google Docs, without external dependencies.

Every deliverable committed to in the approved project proposal was delivered:

| Feature | Status |
|---|---|
| User accounts and profile CRUD | Delivered |
| Event management with recurrence | Delivered |
| Holiday data (both calendars) | Delivered |
| Google OAuth 2.0 login | Delivered |
| Google Calendar export | Delivered |
| Email reminders (SMTP daemon) | Delivered |
| Ethiopian–Gregorian conversion engine (full edge cases) | Delivered |

| | |
|---|---|
| Timezone-aware UTC storage | Delivered |
| Shared events with owner/editor/viewer roles | Delivered |
| Optimistic locking and version conflict detection | Delivered |
| Field-level diff and version history | Delivered |
| Interval merging and time slot suggestion | Delivered |
| Constraint-based slot ranking algorithm | Delivered |
| Figma wireframes | Delivered |
| UML diagrams | Delivered (6 diagrams + 2 use case diagrams) |
| Pytest unit tests | Delivered (116 tests, all passing) |
| Postman integration tests | Delivered (93 tests, all passing) |
| Docker Compose deployment | Delivered |

The codebase is clean, modular, and well-tested. The architecture, three tiers with clear separation of concerns, six focused service classes, and a schema that required zero changes across four sprints demonstrates the kind of upfront engineering discipline that this course aimed to develop.

## 14. Future Plans

| Enhancement | Description | Complexity |
|---|---|---|
| Real-time collaboration via WebSockets | Replace the current optimistic locking model with WebSocket-based live updates. Users would see each other's edits in real time without a page reload. | High |
| Mobile application (React Native) | A React Native version of the frontend would make Zemen far more accessible for everyday use, particularly for Ethiopian users on mobile. | High |
| Natural language event input | Parse phrases like 'Meet John next Tuesday at 2pm Addis Ababa for 1 hour' into structured event data. Planned in an early proposal draft. | High |
| Full bidirectional Google Calendar sync | Extend beyond export-only to full two-way sync: changes in Google Calendar reflect in Zemen and vice versa. | High |
| UI design system | Replace inline CSS with Tailwind CSS or a component library for a consistent, scalable design. | Medium |
| Recurring event series management | Allow editing a single occurrence vs the entire series, with proper recurrence exception handling. | Medium |
| Public holiday API integration | Pull live Ethiopian and Gregorian holiday data from an external API rather than relying on seeded data. | Low |